



Curso: Metodología y gestión de proyectos de Ingeniería Software (IS)

Proyecto: EcoLogistics Web (Sistema Inteligente de Gestión de Residuos)

Responsable Académico: José Luis Martínez Vila

Responsable de Ejecución: Angelina Caballero (Rol:Estudiante/Project Manager)

Año: 2026

TABLA DE CONTENIDOS

1. RESUMEN EJECUTIVO	1
1.1. Propósito del documento	1
1.2. Visión general del producto	1
1.3. Objetivos generales y SMART	1
1.3.1 Objetivo general	1
1.3.2. Objetivos SMART	1
1.4. Alcance del proyecto y entregables	1
1.4.1. Alcance del Minimum Viable Product (MVP)	1
1.4.2. Funcionalidades Incluidas en el Minimum Viable Product (MVP)	2
1.4.1.1. Funcionalidades para Ciudadanos	2
1.4.1.2. Funcionalidades para Conductores	2
1.4.1.3. Funcionalidades para Administradores	2
1.4.1.4. Funcionalidades del Sistema	2
1.4.3 Exclusiones	2
1.4.4. Entregables del Proyecto	2
1.4.4.1. Entregables Obligatorios	2
1.4.3.2. Entregables de Gestión	3
1.4.3.3. Entregables de Análisis	3
1.4.3.4. Entregables de Diseño e Implementación	3
1.4.3.5. Entregables de Seguridad	3
1.4.3.6. Entregables de Calidad y Cierre	3
1.4.5 Criterios de Aceptación del Alcance	3
1.4.6 Límites del Proyecto	4
1.5. Resumen de recursos, plazo y presupuesto	4
1.5.1. Equipo	4
1.5.2. Plazo	4
1.5.3. Decisiones críticas	5
2. INTRODUCCIÓN Y CONTEXTO	6
2.1. Necesidad de negocio	6
2.2 Contexto y justificación	6
2.2.1. Ambito de aplicación	6
2.2.2. Beneficios esperados	7
2.3 Actores y stakeholders	7
2.3.1. Tabla de stakeholders	7
2.3.1. Tabla de Stakeholders Interesados del Proyecto Ecologistics Web	7
2.3.2. Matriz de Influencia de Stakeholders	8
2.3.3. Mapa de interesados ¿?	9
2.4 Objetivos específicos y KPIs	9
2.4.1. Objetivos SMART detallados	9
2.4.2. KPIs operativos	9
2.5 Supuestos y restricciones	10
2.5.1. Supuestos	10
2.5.2. Restricciones	10
3 ORGANIZACIÓN DEL PROYECTO Y GESTIÓN DE RECURSOS HUMANOS	12
3.1 Modelo organizativo y justificación	12
3.2 Organigrama y roles	12
3.2.1. Organigrama gráfico	12
3.2.2. Tabla de roles y responsabilidades	12
3.2.3. Composición de squads y asignación de tareas	13
3.4. Políticas de trabajo y buenas prácticas	15
3.4.1. Ceremonias ágiles	15
3.4.1.1. Daily Scrum (Daily Stand-up)	15
3.4.1.2. Sprint Planning	15

3.4.1.2. Sprint Review	15
3.4.1.2. Sprint Retrospective (Retrospectiva)	15
3.4.2. Pair Programming	16
3.4.3. Control de versiones y branching	16
3.5 Matriz RACI y comunicaciones	16
3.5.1. Matriz RACI	16
3.5.2. Plan de comunicaciones	16
4 INGENIERÍA DE REQUISITOS Y ANÁLISIS	17
4.1 Especificación de Requisitos Software ERS	17
4.1.1 Catálogo de Requisitos Funcionales RF	17
4.1.1.1. Tabla de Requisitos Funcionales (RF)	17
4.1.1.2. Criterios de aceptación	19
4.1.2 Catálogo de Requisitos No Funcionales RNF	19
4.1.2.1 Tabla de Requisitos No Funcionales	19
4.1.3 Matriz de trazabilidad	20
4.1.3.1. Tabla de trazabilidad	21
4.2 Análisis de procesos y datos	22
4.2.1 Diagrama de Flujo de Datos DFD de la gestión de Incidencias	22
4.2.1.1. Diagrama de Flujo de Datos de proceso crítico (DFD)	22
4.2.1.1. DFD proceso crítico de gestion de incidencias y asignación de rutas ..	22
4.2.2 Diccionario de datos conceptual	23
4.2.2.1. Tabla de entidades	23
4.2.2.2. Cardinalidades y justificación	23
4.3 Modelado orientado a objetos y UML	24
4.3.1 Diagrama de Casos de Uso	24
4.3.1.1. Imagen del diagrama y tabla	24
4.3.2 Diagrama de Clases del dominio	25
4.3.2.1. Diagrama conceptual	25
4.3.2.2. Tabla de Clases	25
4.3.3 Diagramas de Secuencia	26
4.3.3.1. Secuencia 1: Ciudadano reporta incidencia	26
4.3.3.2. Secuencia 2: Login conductor y entrega de ruta	26
5 DISEÑO TÉCNICO E IMPLEMENTACIÓN	27
5.1 Selección CASE y entorno de trabajo	27
5.1.1 Upper CASE	27
5.1.1.1. Herramienta elegida	27
5.1.1.2. Instrucciones de uso	27
5.1.2 Lower CASE	27
5.1.2.1 IDE y control de versiones	27
5.1.2.2. Seguridad y carga	28
5.1.3. I-CASE e integración CI/CD	29
5.1.3.1. Objetivo	29
5.1.3.2. Herramienta seleccionada GitHub Actions	29
5.1.3.3. Flujo recomendado del pipeline	29
5.1.3.3. Quality gates y umbrales	29
5.1.3.4. Gestión de secretos y reporting	29
5.2 Arquitectura del software y despliegue	30
5.2.1 Arquitectura lógica y patrón	30
5.2.1.1. Patrón Arquitectónico: MVC / API REST.	30
5.2.1.2. Diagrama lógico	30
5.2.2 Arquitectura de despliegue	30
5.2.2.1. Diagrama de despliegue navegador → CDN → API → BBDD → cache. ..	30
5.2.2.2. Componentes Principales	30
5.2.2.2. Diagrama de Despliegue	30
5.2.3 ADR Elección de Base de Datos	30
5.2.3.1. ADR completo	30
5.3 Diseño de datos físico y APIs	32

5.3.1 Modelo relacional físico	32
5.3.2 Contratos API y ejemplos	33
5.3.2.1. Listado de endpoints	33
5.3.2.2. Autenticación	33
5.4 Lógica crítica y algoritmos	33
5.4.1 Algoritmo de asignación de rutas	33
5.4.1.1. Pseudocódigo	33
5.4.1.2. Parámetros y restricciones	34
5.4.2 Casos de prueba y validación del algoritmo “Asignación de Rutas”	34
5.4.2.1. Tabla de casos de prueba	34
5.4.2.2. Estrategia de validación	35
5.5 UI UX y accesibilidad	35
5.5.1 Mockups y prototipos	35
5.5.1.1. Pantallas	35
5.5.2 Reglas de diseño y criterios WCAG	39
5.5.2.1. Nivel objetivo y reglas.	39
5.5.2.2. Criterios de aceptación de UI	39
5.6 Seguridad y cumplimiento	39
5.6.1 Protección de datos y RGPD	39
5.6.1.1. Políticas	39
5.6.1.2. Medidas técnicas	39
5.6.2 Controles en pipeline y auditoría	40
5.6.2.1. SAST/DAST	40
5.6.2.2. Registro de accesos	40
5.7. Infraestructura y operaciones	40
5.7.1 Estrategia de despliegue y entornos	40
5.7.1.1. Entornos	40
5.7.1.2. Pipelines CI/CD	40
5.7.1.1.1. Diagrama de Arquitectura Web (navegador - servidor - BBDD)	40
5.7.2 Monitorización y runbooks	40
5.7.2.1. Métricas & Runbooks	40
5.7.2.2. Alertas y playbooks	40
Son los pasos de respuesta ante incidentes	40
6 PLANIFICACIÓN, CALIDAD, RIESGOS Y CIERRE	42
6.1.1.WBS - Tabla jerárquica	42
6.1.1.1 Tabla Jerárquica	42
6.1.2.Estimaciones	43
6.1.2.1 Tabla de Estimaciones por horas por tarea y perfil	43
6.2 Cronograma y Diagrama de Gantt	43
6.2.1.Diagrama de Gantt del primer mes	43
6.2.2.Tabla del primer mes (con tareas, inicio, duración, y responsable)	44
6.2.2.Dependencias críticas	44
6.2.2.1. Lista de FI con justificación	44
6.3 Hitos y fechas límite	44
6.3.1.Hito 1	44
6.3.2.Hito 2	44
6.3.3.Hito 3	45
6.4 Plan de pruebas y aseguramiento de calidad	45
6.4.1.Estrategia de pruebas	45
6.4.2.Quality gates	45
6.5 Gestión de riesgos y plan de contingencia	45
6.5.1.Registro de riesgos	45
6.5.2.Planes de contingencia	46
6.6 Gestión del Triángulo de Hierro	46
6.6.1.Opciones ante reducción de plazo	46
6.6.1.1. Opción A – Aumentar recursos	46
6.6.1.2. Opción B – Recortar alcance	46

6.6.1.3. Comparativa	46
6.7 Criterios de aceptación y checklist de entrega	46
6.7.1. Checklist - Lista de verificación para cada entregable	46
6.8 Cierre del proyecto y transferencia	47
6.8.1. Informe final	47
6.8.2. Transferencia	47
7 ANEXOS Y ARTEFACTOS	48
7.1 Anexo A Diagramas UML	48
7.1.1. Clases	48
7.1.1.1. Diagrama de Clases	48
7.1.1.2. Leyenda del Diagrama de Clases	48
7.1.2. Casos de uso	48
7.1.2.1. Diagrama de Casos de Uso	48
7.1.3. Secuencias	49
7.1.3.1. Diagramas de secuencia para reporte de incidencia	49
7.1.3.2. Diagramas de secuencia para generación de ruta	49
7.1.3.3. Diagramas de secuencia para marcación de vaciado	49
7.2 Anexo B Pseudocódigo y pruebas del algoritmo de rutas	50
7.2.1. Pseudocódigo completo	50
7.2.1.1. Algoritmo: Generación y optimización de rutas	50
7.2.1.2. Versión heurística (resumen):	50
7.2.1.3. Comentarios:	50
7.2.2. Casos de prueba	51
7.2.2.1. Tabla de casos de prueba	51
7.3 Anexo C Mockups	51
7.3.1. Capturas	51
7.4 Anexo D Matrices y tablas auxiliares	53
7.4.1. Matriz de trazabilidad	53
7.4.2. Matriz RACI	53
7.4.3. Stakeholders	54
7.5 Anexo E Cronogramas imprimibles	54
7.5.1. Gantt (del primer mes)	54
7.5.2. PERT	55
7.6 Firmas y aprobaciones	55
7.6.1. Plantilla de firmas	55
7.6.2. Registro de aprobaciones	56

1. RESUMEN EJECUTIVO

1.1. Propósito del documento

El propósito de este documento es entregar un Plan de Dirección que permita pasar de la programación a la Ingeniería y producir el MVP operativo de la plataforma EcoLogistics Web.

1.2. Visión general del producto

El proyecto EcoLogistics Web tiene como objetivo desarrollar para el Ayuntamiento una plataforma web para la gestión inteligente de residuos urbanos. La plataforma permite optimizar las rutas de recogida de los contenedores de residuos mediante la monitorización de los camiones de residuos (conductores). También permite a los Usuarios reportar las incidencias que puedan producirse durante todo el proceso de gestión de residuos. Para el desarrollo de la plataforma web de residuos se cuenta con un equipo de 22 personas y un presupuesto de 120.000 euros y plazo de ejecución de 140 horas. La plataforma debe soportar una capacidad de al menos 10.000 usuarios concurrentes.

Principales entregables	1. Project Charter ✓ Organigrama del equipo del proyecto ✓ Justificación de las herramientas utilizadas 2. Carpeta de Análisis ✓ Backlog de Historias de Usuario ✓ Diagramas UML (Clases) ✓ Diagramas de Flujo (Algoritmos). 3. Producto Software ✓ Enlace al repositorio en GitHub con el código fuente estructurado. ✓ Versión desplegada funcional (MVP). 4. Informe de Calidad ✓ Resultado de las pruebas ✓ Manual de Usuario

1.3. Objetivos generales y SMART

1.3.1 Objetivo general

El proyecto EcoLogistics Web tiene como objetivo modernizar la infraestructura de la gestión de residuos urbanos actual para permitir tener un seguimiento real de la situación de los residuos del municipio así como facilitar la interacción con los usuarios del mismo con el fin de mejorar la gestión en el presente y en el futuro.

1.3.2. Objetivos SMART

Los objetivos SMART definidos en la documentación incluyen metas medibles relacionadas con la reducción del tiempo de respuesta, la mejora de la eficiencia operativa y el incremento de la participación ciudadana dentro del proyecto EcoLogistics Web 2.0.

Los KPIs asociados a estos objetivos permiten evaluar el rendimiento del sistema de la plataforma creada en términos de disponibilidad, latencia, calidad de los datos, volumen de incidencias útiles así como la eficacia y eficiencia en la asignación de las rutas.

1.4. Alcance del proyecto y entregables

1.4.1. Alcance del Minimum Viable Product (MVP)

El proyecto EcoLogistics Web 2.0 tiene como objetivo diseñar, analizar y construir un MVP funcional que permita la gestión de residuos urbanos a través de una Plataforma Web para ciudadanos, conductores y administradores municipales.

Se define su alcance mediante los requisitos funcionales (RF) y no funcionales (RNF) de la Especificación de Requisitos del Software, así como los artefactos de análisis (Unified

Modeling Language (UML), Data Flow Diagrams (DFD), y diccionario de datos), la arquitectura propuesta y las restricciones del proyecto (equipo multidisciplinar, metodología híbrida, y 140 horas).

1.4.2. Funcionalidades Incluidas en el Minimum Viable Product (MVP)

1.4.1.1. Funcionalidades para Ciudadanos

- ✓ Registro e inicio de sesión
- ✓ Visualización de contenedores cercanos en mapa
- ✓ Consulta del nivel de llenado de un contenedor
- ✓ Reporte de incidencias (incluye ubicación y foto)
- ✓ Consulta del historial de incidencias enviadas y respuestas

1.4.1.2. Funcionalidades para Conductores

- ✓ Registro e inicio de sesión
- ✓ Recepción de la lista ordenada de las paradas y de la ruta en el mapa
- ✓ Marcado de los contenedores como recogidos (nivel de llenado al 0%)
- ✓ Reporte de incidencias durante la ruta del camión

1.4.1.3. Funcionalidades para Administradores

- ✓ Inicio de sesión (incluye los privilegios correspondientes)
- ✓ Gestión de los conductores (alta, baja, modificación)
- ✓ Gestión de contenedores (alta, baja, modificación)
- ✓ Consulta del estado de las rutas de los camiones (generadas automáticamente)
- ✓ Generación de informes estadísticos y de eficiencia e incidencias

1.4.1.4. Funcionalidades del Sistema

- ✓ Notificaciones para conductores
- ✓ Optimización de rutas basadas en el nivel de llenado de los contenedores (algoritmo >75%)
- ✓ Registro de las auditorías
- ✓ Cumplimiento de los requisitos de seguridad
- ✓ Cumplimiento de los requisitos del Reglamento General de Protección de Datos (RGPD)

1.4.3 Exclusiones

Con el fin de garantizar la entrega del Proyecto dentro del tiempo y con los recursos disponibles se excluyen explícitamente los siguientes elementos:

- ✓ Gestión avanzada de flotas (mantenimiento, combustible, turnos)
- ✓ Automatización de facturación o gestión económica
- ✓ Integración real con sensores reales (se simula el nivel de llenado)
- ✓ Módulos avanzados de analítica predictiva
- ✓ Aplicación móvil nativa (sólo la versión web responsive)
- ✓ Integración con sistemas externos del Ayuntamiento
- ✓ Panel de BI o dashboards avanzados

Las exclusiones de este apartado se consideran que podrían ser ampliadas si así se estima en una futura Fase 3.0.

1.4.4. Entregables del Proyecto

1.4.4.1. Entregables Obligatorios

- ☑ Project Charter
- ☑ Carpeta de Análisis
 - RF y RNF
 - Matriz de trazabilidad
 - UML (Casos de uso, Clases, Secuencia)
 - DFD
 - Diccionario de datos
- ☑ Repositorio con el código fuente

- ☑ MVP desplegado y funcional
- ☑ Informe de Calidad
- ☑ Manual de Usuario
- ☑ Documentación técnica viva (ADRs, tablero visual, decisiones)
- ☑ Informe final del proyecto
- ☑ Firmas de aprobación

1.4.3.2. Entregables de Gestión

- ☑ Project Charter
- ☑ Organigrama del equipo
- ☑ Matriz RACI
- ☑ Matriz de comunicaciones
- ☑ Plan de Gestión del Proyecto

1.4.3.3. Entregables de Análisis

- ☑ Backlog de Historias de Usuario.
- ☑ Especificación de Requisitos Software (RF y RNF).
- ☑ Matriz de trazabilidad.
- ☑ Diagramas UML (Casos de uso, Clases, Secuencia).
- ☑ Diagramas de Flujo de Datos (DFD).
- ☑ Diccionario de datos conceptual.

1.4.3.4. Entregables de Diseño e Implementación

- ☑ Arquitectura del software (MVC + despliegue).
- ☑ ADRs aprobados (Base de Datos, Tablero Visual, Herramientas CASE).
- ☑ Modelo de datos físico.
- ☑ Contratos API.
- ☑ Algoritmo de asignación de rutas (pseudocódigo).
- ☑ Prototipos UI/UX (mockups y wireframes).
- ☑ Código fuente en repositorio GitHub.
- ☑ MVP desplegado funcional.

1.4.3.5. Entregables de Seguridad

- ☑ Informe de Seguridad del Sistema.
- ☑ Resultados de pruebas de seguridad:
 - OWASP ZAP
 - SonarCloud (SAST)
- ☑ Evidencias de corrección de vulnerabilidades críticas.
- ☑ Política de protección de datos y cumplimiento RGPD.
- ☑ Registro de accesos y auditoría.
- ☑ Checklist de seguridad del despliegue.

1.4.3.6. Entregables de Calidad y Cierre

- ☑ Informe de Calidad (unitarias, integración, E2E, carga).
- ☑ Manual de Usuario.
- ☑ Documentación técnica viva (ADRs + tablero visual).
- ☑ Informe final del proyecto.
- ☑ Firmas de aprobación.

1.4.5 Criterios de Aceptación del Alcance

Se considerará que el Proyecto EcoLogistics Web 2.0 estará completado cuando cumpla los siguientes criterios:

- a) EL MVP incluya todas las funcionalidades definidas en el alcance funcional
- b) Todos los Requisitos Must y Should estén implementados y validados
- c) Los Requisitos críticos (rendimiento, seguridad, disponibilidad) se cumplan

- d) El algoritmo de rutas funcione correctamente con datos simulados
- e) El sistema supere las pruebas de seguridad sin vulnerabilidades críticas
- f) El cliente valide la demo de la plataforma final del MVP
- g) Toda la documentación esté entregada y firmada

1.4.6 Límites del Proyecto

- ✓ Tiempo máximo: 140 horas lectivas
- ✓ Equipo fijo: 1 Project Manager (Angelina Caballero) & 21 Desarrolladores
- ✓ Metodología: Metodología Híbrida: Scrum + SXP + Ingeniería CASE
- ✓ Alcance: Cerrado para el MVP (no se aceptan cambios sin replanificación)

1.5. Resumen de recursos, plazo y presupuesto

El presente apartado resume los recursos humanos y técnicos necesarios para la ejecución del proyecto, el plazo disponible para su desarrollo y las decisiones críticas que condicionan la arquitectura y el enfoque técnico del MVP.

1.5.1. Equipo

El proyecto se desarrolla con un equipo multidisciplinar compuesto por perfiles técnicos y de gestión. Disponemos de un equipo 22 personas: 1 product manager y 21 desarrolladores.

La siguiente tabla recoge la distribución de roles y el número de integrantes asignados a cada uno de los perfiles, según la estructura definida en el organigrama y la planificación inicial.

Tabla 1.5.1. Equipo del Proyecto

Rol	Cantidad (personas)	Descripción del rol
Project Manager (PM)	1	Responsable de la planificación, coordinación y seguimiento del proyecto.
Desarrolladores Senior	5	Arquitectura, seguridad, algoritmos complejos y tutoría técnica (XP)
Desarrolladores Junior	10	Desarrollo del grueso de la aplicación web (Frontend/Backend)
Trainees / Estudiantes	6	Maquetación HTML/CSS, documentación, ejecución de pruebas manuales y entrada de datos

1.5.2. Plazo

- ☒ Fecha de Inicio prevista: 28/01/2026
- ☒ Fecha de Entrega prevista: 05/04/2026
- ☒ Duración Estimada: 140 horas lectivas
- ☒ Presupuesto Estimado

Dado que el proyecto se desarrolla en un entorno académico, no existe un presupuesto económico real asociado. No obstante, considerando un escenario profesional con un equipo multidisciplinar y tarifas medias del mercado, el coste estimado para el desarrollo del MVP de Ecologistics Web Fase 2.0 se situaría en torno a 120.000 €. Esta cifra constituye una estimación de alto nivel, adecuada para el resumen ejecutivo, y será ampliada en el apartado correspondiente del documento.

- ☒ Notas Adicionales

Este plazo condiciona el alcance del MVP y obliga a priorizar funcionalidades esenciales, garantizando que el producto final sea funcional, demostrable y evaluable.

1.5.3. Decisiones críticas

Durante las fases iniciales del proyecto se han tomado una serie de decisiones técnicas clave que condicionan la arquitectura, el diseño y la implementación del MVP.

Estas decisiones se han documentado en los ADRs correspondientes y se resumen a continuación:

☒ Base de datos relacional

Se selecciona un modelo relacional por su estructura clara, facilidad de normalización y compatibilidad con los requisitos del sistema.

☒ Patrón arquitectónico MVC / API REST

Permite separar responsabilidades, facilitar el mantenimiento y asegurar la escalabilidad futura.

☒ Stack tecnológico recomendado:

⬆ Backend: Tecnología Node.js & xxxxx (definido en ADR)

⬆ Frontend: Framework React & (definido en ADR)

⬆ BBDD: PostgreSQL (definido en ADR)

⬆ Control de versiones: Git Hub

☒ Uso de herramientas CASE

Para garantizar la trazabilidad, consistencia y calidad de los artefactos del análisis y del diseño

☒ Metodología híbrida(Scrum + XP)

Permite iterar rápidamente, mantener la calidad y también adaptarse a los plazos.

Todas estas decisiones son necesarias con el fin de asegurar la coherencia técnica, reducir los riesgos y garantizar que MVP pueda desarrollarse dentro del tiempo disponible.

2. INTRODUCCIÓN Y CONTEXTO

El presente bloque describe el contexto en el que surge el proyecto ECoLogistics Web 2.0.

También en este bloque se especifican las necesidades de negocio que movitan la creación de la plataforma, los actores implicados, los objetivos específicos, los supuestos, y las restricciones condicionantes de su ejecución. El propósito de esta sección es ofrecer una visión clara del entorno del proyecto así como justifica su relevancia dentro del ámbito de la gestión de residuos urbanos.

2.1. Necesidad de negocio

El sistema actual de gestión de residuos urbanos presenta limitaciones significativas en términos de eficiencia operativa, eficacia del servicio y sostenibilidad económica, generando:

- ✓ La infraestructura costosa y poco eficiente que requiere un mantenimiento elevado y no dispone de mecanismos que permiten supervisar en tiempo el estado de los contenedores
- ✓ Ausencia de planificación dinámica de las rutas de recogida, lo que impide ajustar los recorridos según la demanda real o el nivel de llenado de los contenedores
- ✓ Deficiencia de la capacidad de respuesta ante las incidencias, debido a la falta de información en tiempo real y a la ausencia de herramientas adecuadas que faciliten la trazabilidad completa del proceso
- ✓ Inexistencia de rutas optimizadas, lo que genera desplazamientos innecesarios, mayor consumo de recursos y tiempos de servicio más largos
- ✓ Escasa participación ciudadana, que limita la detección temprana de problemas y reduce la calidad del servicio
- ✓ Generación de retrasos y sobrecostos operativos, derivados de la falta de datos fiables y de la ineficiencia en la gestión diaria
- ✓ Disminución de la calidad del servicio, con contenedores desbordados, incidencias no atendidas y menor percepción de valor por parte de los usuarios
- ✓ Impacto negativo en la sanidad ambiental, al aumentar la probabilidad de acumulación de residuos y focos de suciedad
- ✓ Incremento de quejas ciudadanas, que afecta directamente a la imagen del municipio y a la satisfacción general de la población.

☒ Solución propuesta

EcoLogistics Web surge ante esta necesidad, proporcionando una plataforma digital centralizada de información, optimización de procesos y mejora de la comunicación entre los distintos actores del sistema.

Su arquitectura web propone:

1. Monitorizar el nivel de llenado de contenedores en tiempo real
2. Optimizar dinámicamente las rutas de recogida.
3. Permitir a los Usuarios reportar incidencias (contenedores rotos, basura fuera).

☒ Beneficios esperados:

Mejorar la gestión de residuos urbanos:

Optimizar la recogida de residuos (evitar camiones semivacíos)

Optimizar el coste de las rutas (gasolina desperdiciada)

Optimizar el mobiliario urbano (evitar contenedores rotos)

Mejorar la sanidad ambiental (insalubridad)

Mejorar la valoración ciudadana (tardanza en la recogida de basura)

2.2 Contexto y justificación

2.2.1. Ambito de aplicación

El proyecto se desarrolla en un contexto donde los municipios buscan modernizar sus servicios públicos mediante soluciones digitales accesibles, sostenibles y escalables.

La gestión inteligente de residuos es un área que impacta directamente en la calidad de vida de los ciudadanos y en la eficiencia de los servicios municipales.

Se justifica La plataforma EcoLogistics Web 2.0 por los siguientes motivos:

- ✓ Permite digitalizar los procesos manuales y reducir errores operativos
- ✓ Facilita la optimización de rutas, disminuyendo los tiempos y costos del servicio
- ✓ Mejora la experiencia del ciudadano, que puede reportar las incidencias fácilmente
- ✓ Proporciona datos estadísticos estructurados para posteriores análisis e informes
- ✓ El servicio se alinea con estrategias ecológicas de Ciudad Inteligente (Smart City)

El MVP desarrollado en este proyecto, constituye un primer paso hacia una futura plataforma completa de gestión inteligente de residuos urbanos.

2.2.2. Beneficios esperados

Los principales beneficios esperados por la puesta en funcionamiento de la plataforma EcoLogistics Web 2.0 son los siguientes:

- ✓ Optimización del costo de la recolección de residuos
- ✓ Optimización de la recolección de residuos
- ✓ Mejora de la sanidad ambiental
- ✓ Fomento de la economía circular
- ✓ Reducción de la huella de carbono
- ✓ Toma de decisiones basada en datos
- ✓ Transparencia y confianza
- ✓ Respuesta más rápida ante emergencias
- ✓ Optimización del mobiliario urbano

2.3 Actores y stakeholders

El proyecto EcoLogistics Web 2.0 involucra a un conjunto diverso de actores que participan en su desarrollo, validación y uso.

Para garantizar una gestión adecuada, los stakeholders se clasifican en internos y externos, según su relación con el proyecto y su nivel de implicación con el mismo.

☒ Stakeholders Internos

Son aquellos que participan directamente en la ejecución, validación o dirección del proyecto.

☒ Stakeholders Externos

Son aquellos que no participan explícitamente en el desarrollo, sin embargo utilizan el sistema o se ven afectados por su funcionamiento.

2.3.1. Tabla de stakeholders

2.3.1. Tabla de Stakeholders Interesados del Proyecto EcoLogistics Web

Stakeholder	Tipo	Interés
Product Owner (Responsable Académico)	Interno	Representa la voz del cliente, prioriza requisitos, valida entregables, asegura MVP cumple objetivos funcionales y formativos establecidos
Jefe/a de Proyecto	Interno	Coordina el trabajo del equipo, se asegura de que el trabajo se entregue a tiempo, dentro del presupuesto, con la calidad acordada, y gestiona los riesgos.
Equipo de Frontend (UI/UX)	Interno	Diseñan una interfaz que resulte clara, accesible, coherente con los requisitos y agradable de usar.
Equipo de Backend (API)	Interno	Construyen una API REST sólida, segura, escalable y con buen rendimiento.

Equipo de Base de Datos y DevOps	Interno	Definen un modelo eficiente, optimizan las consultas y mantienen la infraestructura y los despliegues en buen estado
Equipo de QA / Testing	Interno	Realizan pruebas de todo tipo para garantizar la estabilidad y seguridad del sistema
Equipo de Documentación	Interno	Elaboran guías de usuario y manuales, y documentación técnica clara y estructurada
Ayuntamiento (cliente)	Externo	Busca optimizar la gestión de residuos, reducir los costes y mejorar la eficiencia del servicio
Ciudadanos	Externo	Consultan contenedores, reportan incidencias y contribuyen a la mejora del servicio
Conductores de camiones	Externo	Requieren una aplicación clara y usable durante la ruta, con recorridos eficientes
Administrador/gestor de flota	Externo	Gestiona rutas, conductores, contenedores e incidencias, y necesita datos fiables para la toma de decisiones
Personal de mantenimiento	Externo	Recibe avisos sobre contenedores dañados o incidencias que requieren intervención física.
Policía Local	Externo	Supervisa que los contenedores no generen riesgos, obstrucciones o problemas de seguridad vial
Servicios urbanos (limpieza viaria)	Externo	Buscan mejorar la percepción ciudadana y reducir el número de quejas
Alcaldía / Concejales	Externo	Interesados en mejorar la imagen del municipio, la satisfacción ciudadana y la eficiencia del servicio
Comunidad local / población afectada	Externo	percibe la calidad del servicio, la salubridad urbana y el impacto de la gestión de residuos.

2.3.2. Matriz de Influencia de Stakeholders

Stakeholder	Tipo	Influencia	Interés	Estrategia de gestión
Product Owner (Responsable Académico)	Interno	Alta	Alta	Gestionar de cerca
Jefe/a de Proyecto	Interno	Alta	Alta	Gestionar de cerca
Equipo Frontend (UI/UX)	Interno	Media	Alta	Mantener involucrados
Equipo Backend (API)	Interno	Media	Alta	Mantener involucrados
Equipo BD / DevOps	Interno	Media	Media	Consultar cuando sea necesario
Equipo QA / Testing	Interno	Media	Alta	Mantener involucrados
Equipo de Documentación	Interno	Baja	Media	Mantener informados
Ayuntamiento (cliente)	Externo	Alta	Alta	Gestionar de cerca
Administrador	Externo	Media	Alta	Mantener satisfechos
Gestor de flota	Externo	Baja	Alta	Mantener informados
Conductores de camiones	Externo	Baja	Media	Mantener informados
Ciudadanos	Externo	Baja	Media	Mantener informados
Personal de mantenimiento	Externo	Baja	Media	Mantener informados

Servicios urbanos (limpieza viaria)	Externo	Media	Media	Consultar cuando sea necesario
Policía Local	Externo	Media	Baja	Monitorizar
Alcaldía / Concejales	Externo	Alta	Media	Mantener satisfechos
Comunidad local / población afectada	Externo	Baja	Baja	Monitorizar

2.3.3. Mapa de interesados ¿?

El mapa de interesados representa la posición de cada stakeholder según su nivel de influencia sobre el proyecto y su grado de interés en los resultados del mismo. Esta clasificación permite definir estrategias de gestión adecuadas para cada grupo, optimizando la comunicación y la toma de decisiones

indicación para insertar gráfico.

2.4 Objetivos específicos y KPIs

2.4.1. Objetivos SMART detallados

☒ Específicos (Specific)

- Reducir en un 30% el tiempo de respuesta ante contenedores llenos mediante el uso de rutas optimizadas y notificaciones automáticas a los conductores, alcanzándolo en un plazo máximo de seis meses tras el lanzamiento de la plataforma.
- Incrementar en un 40% la proporción de incidencias correctamente reportadas gracias a un formulario más simple y a su integración con el mapa interactivo, logrando este resultado durante el primer año de operación continua.

☒ Medibles (Measurable)

- Disminuir en un 30% el tiempo medio que transcurre desde que un contenedor supera el 75% de llenado hasta que es vaciado.
- Incrementar en un 30% el número de incidencias válidas registradas por la ciudadanía (contenedor lleno, dañado, etc.), tomando como referencia el volumen de reportes correctos del periodo previo al lanzamiento de la plataforma.

☒ Alcanzables (Achievable)

- Gracias al uso del algoritmo de optimización de rutas y al envío automático de avisos a los conductores, la reducción del tiempo de respuesta es viable dentro de los recursos y capacidades del sistema.
- Mediante la simplificación del formulario de incidencias y su integración con el mapa interactivo, el aumento de reportes válidos es factible sin requerir cambios estructurales ni inversiones adicionales.

☒ Relevantes (Relevant)

- Impacta directamente en la limpieza urbana y en la satisfacción de la ciudadanía, al mejorar la rapidez y eficacia del servicio de recogida.
- Refuerza la participación ciudadana y la detección temprana de incidencias, permitiendo actuar antes y con mayor precisión sobre los problemas del entorno urbano.

☒ Temporales (Time-bound)

- En los 6 meses siguientes al lanzamiento de la plataforma, para los objetivos relacionados con la reducción del tiempo de respuesta.
- En el primer año de operación continua, para los objetivos vinculados al incremento de incidencias válidas registradas.

2.4.2. KPIs operativos

KPIs operativos: tabla con definición, fórmula y frecuencia de medición.

Para garantizar el seguimiento del impacto real del proyecto una vez desplegado, se establecen los siguientes Indicadores Clave de Desempeño KPIs.

Esta sección define la unidad de medida y la meta asociada a cada objetivo.

☒ Específicos

- KPI 1 – Tiempo de respuesta ante contenedores llenos

Reducción del 30% en el tiempo medio de actuación durante los primeros 6 meses tras el lanzamiento, gracias a rutas optimizadas y alertas automáticas.

- KPI 2 – Tasa de reportes ciudadanos válidos

Incrementar la tasa de reportes de ciudadanos útiles en un 40% con relación a la cantidad de reportes útiles recibidos en la actualidad.

☒ Medibles

- KPI 1 – Tiempo de respuesta ante contenedores llenos

Reducir en un 30% el tiempo medio entre que un contenedor supera el 75% de llenado y su vaciado efectivo.

- KPI 2 – Incidencias ciudadanas válidas registradas

Incrementar en un 30% el número de incidencias válidas registradas por la ciudadanía (contenedor lleno, dañado, etc.), tomando como referencia el volumen de reportes correctos del periodo previo al lanzamiento de la plataforma.

☒ Alcanzables

- KPI 1 – Eficiencia en la respuesta a contenedores llenos

Mide el impacto real del algoritmo de optimización de rutas y de las notificaciones automáticas.

- KPI 2 – Calidad del reporte de incidencias ciudadanas

Mide el efecto de simplificar el formulario y de integrarlo con el mapa interactivo.

☒ Relevantes

- KPI 1 – Tiempo de respuesta ante contenedores lleno

Mide el tiempo medio de respuesta (en minutos u horas) desde que un contenedor supera el 75% de llenado hasta su vaciado.

- KPI 2 – Incidencias válidas registradas por la ciudadanía

Mide el número de incidencias válidas registradas (contenedor lleno, dañado, etc.)

☒ Temporales

- KPI 1 – Evolución del tiempo medio de respuesta en el periodo de 6 meses

Medición mensual del tiempo medio de respuesta desde el 75% de llenado hasta el vaciado.

- KPI 2 – Evolución del número de incidencias válidas en el primer año

Mide el número de incidencias válidas registradas por mes.

2.5 Supuestos y restricciones

Este proyecto se desarrolla bajo una serie de supuestos y restricciones que condicionan su alcance y ejecución.

2.5.1. Supuestos

Relación de supuestos

- ✓ Los datos de nivel de llenado de contenedores se simulan, ya que no existe integración real con sensores en tiempo real (IoT).
- ✓ Los usuarios finales (ciudadanos, conductores y administradores) se representarán mediante perfiles simulados.
- ✓ El entorno de despliegue principal será local o basado en servicio gratuitos
- ✓ El equipo mantiene disponibilidad constante durante las horas lectivas asignadas.

2.5.2. Restricciones

Relación de restricciones

- ✓ El proyecto debe completarse en un máximo de 140 horas lectivas

- ✓ El equipo está compuesto por un Project Manager (PM) y 21 desarrolladores con distintos niveles de experiencia.
- ✓ El alcance de MVP es cerrado y no admite ampliaciones sin replanificación.
- ✓ Se deben utilizar herramientas CASE para garantizar trazabilidad y consistencia.
- ✓ La solución debe cumplir requisitos básicos de seguridad y protección de datos.

Estas condiciones aseguran que el proyecto se mantenga dentro de un marco realista y controlado, permitiendo entregar un MVP funcional y evaluable.

3 ORGANIZACIÓN DEL PROYECTO Y GESTIÓN DE RECURSOS HUMANOS

3.1 Modelo organizativo y justificación

El proyecto EcoLogistics Web 2.0 adopta un modelo organizativo basado en squads multidisciplinares, inspirado en marcos ágiles modernos (Scrum & DevOps). Este enfoque facilita distribuir el trabajo por áreas funcionales, mejorar la autonomía de los equipos y acelerar la entrega de valor.

Los squads se han seleccionado por los siguientes motivos:

- ✓ Especialización funcional

Cada squad se centra en área clave: Frontend, Backend, BBDD, QA & Seguridad, Documentación.

- ✓ Autonomía Operativa

Los equipos pueden avanzar en paralelo sin dependencias críticas

- ✓ Mejor comunicación interna

Cada squad tiene un alcance claro y un responsable técnico

- ✓ Alineación con prácticas ágiles

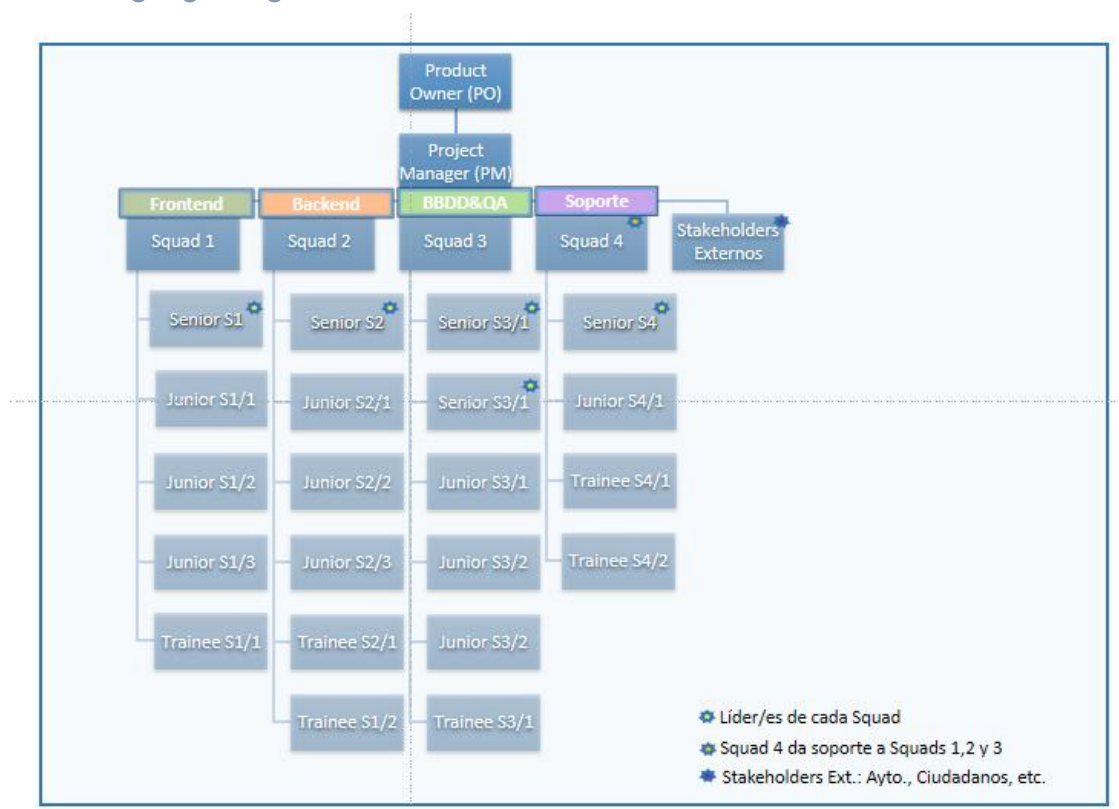
Facilita ceremonias ágiles, iteraciones y entregas incrementales

- ✓ Trazabilidad y control

Cada Squad tiene entregables definidos y medibles

3.2 Organigrama y roles

3.2.1. Organigrama gráfico



3.2.2. Tabla de roles y responsabilidades

Perfil	Rol	Responsabilidades
Responsable Académico	Product Owner (PO)	Priorización del backlog, validación de entregables, definición de criterios de aceptación
Estudiante Angelina	Jefe de Proyecto (PM)	Planificación, seguimiento, gestión de riesgos, coordinación entre squads

Squad 1	Frontend (UI/UX)	Diseño de interfaz, accesibilidad, implementación de vistas y componentes
Squad 2	Backend (API)	Desarrollo de API REST, lógica de negocio, seguridad y rendimiento
Squad 3	BD/DevOps	Modelado de datos, optimización de consultas, despliegues, CI/CD
Squad 3	QA/Testing	Pruebas funcionales, integración, regresión, seguridad
Squad 4	Soporte Document.	Manuales, guías de usuario, documentación técnica

3.2.3. Composición de squads y asignación de tareas

Squad 1 Este equipo se dedica a trabajar en el Frontend (el interfaz de ciudadano y conductor) Entregables: * Prototipos UI * Componentes React * Flujos de navegación		
Perfil	Rol	Responsabilidades
Senior S1	Full Stack	Define los estándares de accesibilidad (WCAG) y las guías de diseño. Supervisa la arquitectura del frontend y la optimización del rendimiento visual.
Junior S1/1	Front-end	Implementa componentes reutilizables y accesibles. Garantiza que el diseño sea responsive en distintos dispositivos.
Junior S1/2	Front-end	Desarrolla la lógica de interacción del usuario (formularios, validaciones, navegación). Optimiza tiempos de carga y reduce el uso innecesario de recursos.
Junior S1/3	Front-end	Construye prototipos interactivos y anima flujos clave (transiciones, feedback visual). Mejora la usabilidad mediante pruebas con usuarios.
Trainee S1	Soporte	Realiza pruebas manuales de interfaz (UI testing). Documenta mejoras de usabilidad y propone ajustes visuales.
Squad 2 Este equipo se dedica a trabajar en el Backend (la lógica de las rutas, las API) Entregables: * API funcional documentada * Servicios de negocio * Pruebas unitarias		
Perfil	Rol	Responsabilidades
Senior S2	Full Stack	Diseña la arquitectura de servicios y define las reglas de negocio principales. Supervisa la calidad del código backend y los patrones de diseño.
Junior S2/1	Back-end	Implementa microservicios internos y gestiona la comunicación entre ellos. Optimiza consultas y operaciones intensivas del servidor.
Junior S2/2	Back-end	Desarrolla servicios de autenticación, autorización y gestión de sesiones. Mantiene la lógica de negocio relacionada con

		usuarios y permisos.
Junior S2/3	Back-end	Implementa servicios de notificaciones (email, push, avisos internos). Gestiona colas de mensajes y tareas asíncronas.
Trainee S2/1	Soporte	Realiza pruebas de API con herramientas como Postman. Documenta endpoints y flujos de negocio.
Trainee S2/2	Soporte	Apoya en la creación de mocks y datos de prueba. Participa en pair programming para aprender patrones backend.
Squad 3 Este equipo se dedica a gestionar la base de datos, la arquitectura, la seguridad y la calidad de la plataforma Entregables: * Pipeline CI/CD * Informes de pruebas * Auditoría de seguridad		
Perfil	Rol	Responsabilidades
Senior S3/1	DevOps	Diseña la infraestructura en la nube, pipelines CI/CD y monitorización. Define políticas de despliegue y rollback.
Senior S3/2	Seguridad & Calidad	Establece controles de seguridad, cifrado y auditoría. Supervisa la calidad del software mediante métricas y revisiones.
Junior S3/1	DevOps	Automatiza despliegues, backups y tareas de mantenimiento. Configura herramientas de observabilidad (logs, métricas, alertas).
Junior S3/2	Pruebas	Implementa pruebas de carga, estrés y rendimiento. Evalúa la estabilidad del sistema bajo alta demanda.
Junior S3/3	Pruebas	Desarrolla pruebas automatizadas de regresión y smoke tests. Mantiene suites de testing y reporta métricas de calidad.
Trainee S3/1	Soporte	Documenta procesos de infraestructura y seguridad. Ayuda en tareas básicas de mantenimiento.
Trainee S3/1	Soporte	Colabora en la ejecución de pruebas manuales y validación de entornos. Apoya en la gestión de incidencias técnicas.
Squad 4 Este equipo se dedica a gestionar el mantenimiento, soporte y la documentación de la plataforma Entregables: * Manual de usuario * Documentación técnica * Guías de despliegue		
Perfil	Rol	Responsabilidades
Senior S4	Full Stack	Define la estrategia de documentación y soporte. Supervisa la calidad de los manuales, guías y procedimientos.
Junior S4/1	Back-end Front-end	Desarrolla herramientas internas para soporte y mantenimiento. Automatiza tareas repetitivas del equipo de soporte.

Trainee S4/1	Soporte	Atiende incidencias de primer nivel y documenta soluciones. Colabora en la creación de guías para usuarios y técnicos.
Trainee S4/2	Soporte	Realiza pruebas de mantenimiento y verifica que las correcciones funcionen. Mantiene actualizado el repositorio de documentación.

3.4. Políticas de trabajo y buenas prácticas

3.4.1. Ceremonias ágiles

Las ceremonias ágiles estructuran el trabajo del equipo para cada sprint. Estas reuniones garantizan la transparencia, alineación y mejora continua. En este proyecto se aplican las ceremonias propias de Scrum, adaptadas al modelo de squads multidisciplinares.

3.4.1.1. Daily Scrum (Daily Stand-up)

Reunión cerrada (timed-box) de 15 minutos, siempre a la misma hora.

Cada miembro del equipo responde a tres puntos clave:

- Que hice ayer
- Qué haré hoy
- Que impedimentos tengo

Su objetivo es diariamente sincronizar al equipo y detectar cualquier bloqueo.

Durante la reunión en un primer momento no se resuelven problemas, tan sólo se identifican. Si existieran impedimentos, el Scrum Master (o PM si corresponde) los gestiona con posterioridad.

El resultado que se busca es la detección temprana de los riesgos durante el sprint y la visibilidad del progreso del equipo.

3.4.1.2. Sprint Planning

Reunión de 4 horas para un sprint de 2 semanas (2 horas para 1 semana) establecida al inicio de cada Sprint.

En la reunión el Product Owner presenta los elementos priorizados de backlog para el Sprint. El equipo analiza, estima y selecciona aquellos elementos del backlog con los que puede comprometerse a entregar al final del Sprint.

Como resultado de la reunión se define el objetivo común o Sprint Goal y el incremento del producto a entregar al final del Sprint o Sprint Backlog.

3.4.1.2. Sprint Review

Reunión de 2 horas para un sprint de 2 semanas (1 hora para 1 semana) establecida al final de cada Sprint (justo antes de terminar el ciclo).

En la reunión el Product Owner y los stakeholders validan las Historias de Usuario que cumplen los criterios de aceptación (DOD). El equipo demuestra el incremento funcional que ha completado basado en el backlog establecido en el Sprint Planning. Además de la presentación teórica, si existe software (producto) operativo se muestra. El incremento del producto es validado y el backlog se actualiza debidamente según el feedback real de Product Owner y Stakeholders.

3.4.1.2. Sprint Retrospective (Retrospectiva)

Reunión de 1,5 a 2 horas para un sprint de 2 semanas (1 hora para 1 semana) establecida al final de cada Sprint (justo antes de terminar el ciclo).

En la reunión el Equipo al completo se reúne (PO+Desarrolladores+SM) con el fin de analizar que ha funcionado, que no ha funcionado y que se puede mejorar en relación con el Sprint que se ha terminado. Se considera un espacio seguro para poder hablar con transparencia. Se identifican las acciones concretas para el siguiente Sprint. Se revisan las métricas internas (velocidad, bloqueos, defectos).

- 3.4.2. Pair Programming
- 3.4.3. Control de versiones y branching
- 3.5 Matriz RACI y comunicaciones
 - 3.5.1. Matriz RACI
 - 3.5.2. Plan de comunicaciones

4 INGENIERÍA DE REQUISITOS Y ANÁLISIS

4.1 Especificación de Requisitos Software ERS

La Especificación de Requisitos Software (ERS) de manera estructurada define los requisitos funcionales del sistema EcoLogistics Web 2.0.

Estos requisitos funcionales (RF) y requisitos no funcionales (RNF) del sistema se han obtenido mediante el análisis del dominio, la revisión de documentación previa y el modelado de los procesos.

4.1.1 Catálogo de Requisitos Funcionales RF

A continuación incluimos los RF de la plataforma EcoLogistics Web 2.0.

4.1.1.1. Tabla de Requisitos Funcionales (RF)

Tabla organizada por actor

ID	Requisito	Descripción	Prioridad	Criterio de aceptación
A. Requisitos del Actor Usuario/Ciudadano				
RF-001	Registro de Usuario	El sistema debe permitir que un Usuario se registre proporcionando nombre, email y contraseña	Must	CA1: Validación de email CA2: Contraseña segura CA3: Confirmación de registro
RF-002	Inicio de sesión	El Usuario debe poder iniciar sesión con sus credenciales (email y contraseña)	Must	CA1: Validación de credenciales CA2: Mensaje de error si son incorrectas
RF-003	Geolocalización de contenedores	El sistema debe mostrar en un mapa interactivo los contenedores cercanos a la ubicación del Usuario.	Should	CA1: Mapa interactivo CA2: Filtro por tipo de contenedor
RF-004	Consultar el estado del contenedor	El Usuario podrá ver el nivel de llenado actual de un contenedor específico.	Should	CA1: Nivel de llenado visible CA2: Última fecha de vaciado
RF-005	Reporte de incidencias	El Usuario podrá reportar una incidencia seleccionando un contenedor e indicar el tipo de problema del contenedor	Must	CA1: Selección de contenedor CA2: Tipo de incidencia CA3: Registro en estado "Pendiente"
RF-006	Historico de incidencias	El Usuario podrá consultar el listado de incidencias que ha reportado, con su estado actual (pendiente, en proceso, resuelta)	Should	CA1: Listado con estados CA2: Filtro por fecha
RF-007	Adjuntar foto en incidencia	El Usuario podrá adjuntar una imagen opcional al reportar una incidencia	Should	CA1: Subida de imagen CA2: Vista previa
RF-008	Notificación de estado de incidencia	El Usuario recibirá notificaciones cuando su incidencia cambie de estado	Could	CA1: Notificación push/email CA2: Mensaje con fecha de

				resolución
B. Requisitos del Actor Conductor				
RF-009	Registro del conductor	El sistema debe permitir que un Conductor se registre proporcionando nombre, matrícula de vehículo	Should	CA1: Validación de matrícula CA2: Datos obligatorios
RF-010	Inicio de sesión	El Conductor debe poder iniciar sesión con sus credenciales	Must	CA1: Validación de credenciales CA2: Acceso a panel de rutas
RF-011	Visualización de la ruta asignada	El conductor podrá ver la ruta del día con el listado ordenado de contenedores a visitar y su ubicación en el mapa	Must	CA1: Mapa con paradas CA2: Orden optimizado
RF-012	Marcar el contenedor como recogido	El conductor podrá indicar que ha vaciado un contenedor actualizando su nivel de llenado a 0%	Must	CA1: Nivel de llenado pasa a 0% CA2: Registro de fecha/hora
RF-013	Reportar incidencia en ruta	El conductor podrá reportar incidencias durante la ruta (contenedor dañado, imposibilidad de acceso, etc.)	Should	CA1: Selección de contenedor CA2: Tipo de incidencia
RF-014	Detalles del contenedor durante la ruta	Al seleccionar un contenedor de la ruta, el conductor podrá los detalles del contenedor (la dirección, el tipo de contenedor, el nivel de llenado)	Should	CA1: Información completa CA2: Datos actualizados
RF-015	Recibir notificaciones de aviso y detalles del contenedor	El conductor recibirá notificaciones cuando un contenedor supere el 75% de llenado	Could	CA1: Notificación push CA2: Identificación del contenedor
RF-016	Cerrar incidencia con evidencia	El Conductor podrá cerrar una incidencia adjuntando foto o comentario	Should	CA1: Adjuntar evidencia CA2: Cambio a estado "Resuelta"
RF-017	Confirmar finalización de ruta	El Conductor podrá marcar la ruta como completada	Could	CA1: Botón de finalización CA2: Registro de hora
C. Requisitos del Actor Administrador				
RF-018	Inicio de sesión	El Administrador debe poder iniciar sesión con sus credenciales privilegiadas	Must	CA1: Validación de rol CA2: Acceso a panel de gestión
RF-019	Gestión de conductores	El administrador podrá dar de alta, modificar y dar de baja conductores en el sistema	Must	CA1: CRUD completo CA2: Validación de datos
RF-020	Asignación de rutas	El administrador podrá asignar a los conductores las rutas generadas automáticamente	Must	CA1: Selección de conductor CA2: Notificación al conductor
RF-021	Gestión de	El administrador podrá dar de	Must	CA1: CRUD

	contenedores	alta, modificar y dar de baja los contenedores en el sistema		completo CA2: Validación de ubicación
RF-022	Consultar estado de contenedores	El administrador podrá consultar el estado y los detalles de todos los contenedores (nivel de llenado, última recogida, etc.)	Should	CA1: Datos actualizados CA2: Filtros avanzados
RF-023	Generación de informes	El administrador podrá generar informes sobre eficiencia de rutas, de contenedores, gestión de incidencias	Could	CA1: Exportación PDF/CSV CA2: Filtros por fecha
RF-024	Gestión de incidencias	El Administrador podrá ver, filtrar y reasignar incidencias	Should	CA1: Filtros por estado CA2: Reasignación de responsable
RF-025	Gestión de usuarios	El Administrador podrá gestionar permisos y roles	Could	CA1: Asignación de rol CA2: Control de acceso
D. Requisitos del Sistema				
RF-026	Autenticación unificada	El sistema debe gestionar autenticación para todos los actores mediante un módulo común	Must	CA1: JWT/HTTPS CA2: Control de sesiones
RF-027	Registro de auditoría	El sistema debe almacenar un log de acciones críticas (login, cambios, incidencias)	Should	CA1: Log seguro CA2: Consulta por administrador

4.1.1.2. Criterios de aceptación

Los criterios de aceptación definen las condiciones verificables que deben cumplir cada funcionalidad para considerarse correctamente implementada dentro del sistema de gestión de residuos.

Estos criterios están integrados en la Tabla de Requisitos Funcionales, permiten calificar de forma objetiva el comportamiento esperado en procesos clave como la comunicación de incidencias, la gestión de rutas, la interacción de los conductores y los usuarios.

Su formulación garantiza que cada requisito pueda evaluarse mediante pruebas funcionales claras, manteniendo la trazabilidad con las historias de usuario (US) y sirviendo como base para el modelado posterior del sistema, los casos de uso y los diagramas UML incluidos en este proyecto.

4.1.2 Catálogo de Requisitos No Funcionales RNF

Los Requisitos No Funcionales definen las propiedades de calidad del sistema, tales como el rendimiento, la seguridad, disponibilidad, usabilidad y mantenibilidad.

Estos requisitos complementan a los requisitos funcionales y garantizan que el sistema cumpla con los estándares adecuados para su operación en un entorno como el Ayuntamiento.

4.1.2.1 Tabla de Requisitos No Funcionales

A continuación se presenta la tabla De RNF, organizada por categoría, con su métrica objetivo y prioridad.

4.1.2.1 Tabla de Catálogo de Requisitos No Funcionales (RNF)

ID	Categoría	Descripción	Métrica	Prioridad
----	-----------	-------------	---------	-----------

Requisitos por Usuario/Ciudadano				
RNF-001	Portabilidad	La aplicación debe funcionar correctamente en las últimas versiones de los principales buscadores	Compatibilidad de con ultima versión de navegadores	Alta
RNF-002	Rendimiento	El mapa con los contenedores debe cargar en menos de 2 segundos	Tiempo de carga del mapa (inf. 2 min)	Alta
RNF-003	Usabilidad	La interfaz web debe cumplir con las directrices WCAG	Accesibilidad Cumplimiento WCAG 2.1	Media
Requisitos por Conductor				
RNF-004	Disponibilidad	El sistema tendrá una disponibilidad del 99.5% en horario de servicio	Tiempo de actividad	Alta
RNF-005	Concurrencia	El sistema debe soportar una capacidad de 10.000 usuarios simultáneos con respuesta inferior a 3 seg	Usuarios simultáneos	Alta
Requisitos por Ayuntamiento				
RNF-006	Disponibilidad	El sistema tendrá una disponibilidad del 99.5% en horario de servicio	Tiempo de actividad	Alta
RNF-007	Seguridad	Todas las contraseñas de usuarios se almacenarán en la base de datos utilizando encriptación	Encriptación de contraseñas	Alta
RNF-008	Escalabilidad	La arquitectura debe permitir añadir nuevas instancias del backend sin modificar el código	Escalado horizontal	Media
RNF-009	Privacidad (GDPR)	El sistema debe cumplir con el Reglamento General de Protección de Datos en el tratamiento de datos personales	Derecho al olvido (consentimiento explícito)	Alta
Requisitos del Equipo de Seguridad				
RNF-010	Seguridad	Todas las contraseñas de usuarios se almacenarán en la base de datos utilizando encriptación	Encriptación de contraseñas	Alta
RNF-011	Mantenibilidad	El código fuente deberá mantener una cobertura de pruebas superior al 80% y pasar los tests de calidad	Calidad del código	Alta
RNF-012	Auditoría/Logging	El sistema debe registrar eventos críticos (login, incidencias, cambios de estado) para auditoría	Eventos crítico Registrados (100%)	Alta

4.1.3 Matriz de trazabilidad

La matriz de trazabilidad garantiza la relación entre los requisitos funcionales (RF), las historias de usuario (US), los diagramas UML y las tareas de desarrollo. Permite verificar

que cada requisito está cubierto por un caso de uso y que existe una implementación asociada y validada.

Establecemos la siguiente definición de los estados, alineados con las fases de desarrollo:

Estado	Definición
Pendiente	El requisito está identificado pero aún no se ha comenzado a trabajar
En análisis	Se está detallando el requisito y validando con el cliente. También se están definiendo los criterios de aceptación
En diseño	Se está diseñando la solución (diagramas, arquitectura, interfaz) antes de codificar
En desarrollo	El equipo está escribiendo el código y las pruebas correspondientes
En pruebas	El código está implementado y se están ejecutando pruebas (unitarias, integración, y aceptación)
Validado	El requisito ha sido probado y cumple los criterios de aceptación. Está listo para ser entregado
Cerrado	El requisito ha sido entregado y aceptado formalmente

4.1.3.1. Tabla de trazabilidad

ID Requisito	Historia de Usuario	Diagrama asociado	Tarea de desarrollo	Estado
RF-001	—	CU-Login	FE-Login / BE-Auth	Pendiente
RF-002	—	CU-Login	FE-Login / BE-Auth	Pendiente
RF-003	—	CU-Mapa	FE-Mapa / BE-Contenedores	Pendiente
RF-004	—	CU-Mapa	FE-DetalleContenedor	Pendiente
RF-005	US01	CU01 Reportar incidencia	FE-Incidencias / BE-Incidencias	Pendiente
RF-006	—	CU01	FE-Historial	Pendiente
RF-007	US01	CU01	FE-Upload	Pendiente
RF-008	US02	CU02 Notificación	BE-Notificaciones	Pendiente
RF-009	—	CU-Conductor	BE-Conductores	Pendiente
RF-010	—	CU-Conductor	FE-LoginConductor	Pendiente
RF-011	US04	CU04 Ruta optimizada	FE-Ruta / BE-Rutas	Pendiente
RF-012	US03	CU03 Marcar recogida	FE-Recogida / BE-Recogida	Pendiente
RF-013	—	CU03	FE-IncidenciaConductor	Pendiente
RF-014	—	CU-Contenedor	FE-DetalleContenedor	Pendiente
RF-015	—	CU-Notificaciones	BE-Alertas	Pendiente
RF-016	—	CU-Incidencias	FE-CerrarIncidencia	Pendiente
RF-017	—	CU-Ruta	FE-FinalizarRuta	Pendiente
RF-018	—	CU-AdminLogin	FE-AdminLogin	Pendiente
RF-019	US06	CU06 Gestión conductores	FE-Conductores / BE-Conductores	Pendiente
RF-020	US05	CU05 Asignar ruta	FE-Rutas / BE-Rutas	Pendiente
RF-021	—	CU-Contenedores	FE-Contenedores	Pendiente
RF-022	—	CU-Contenedores	FE-Dashboard	Pendiente
RF-023	—	CU-Informes	BE-Informes	Pendiente
RF-024	—	CU-Incidencias	FE-IncidenciasAdmin	Pendiente
RF-025	—	CU-Usuarios	FE-Usuarios	Pendiente

RF-026	—	CU-Auth	BE-Auth	Pendiente
RF-027	—	CU-Auditoría	BE-Logs	Pendiente

4.2 Análisis de procesos y datos

4.2.1 Diagrama de Flujo de Datos DFD de la gestión de Incidencias

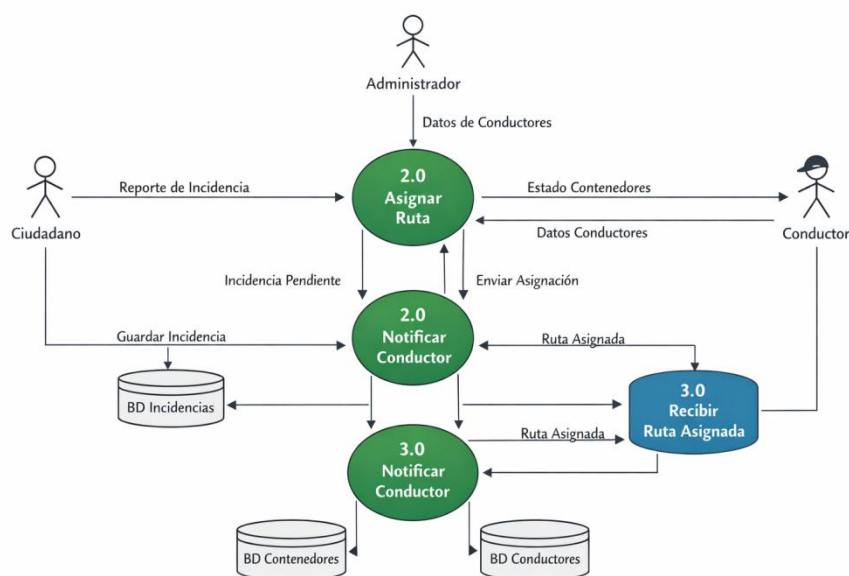
4.2.1.1. Diagrama de Flujo de Datos de proceso crítico (DFD)

Título del Proceso Crítico: Gestion de incidencias y Asignación de rutas

A continuación se presenta el DFD del proceso crítico que integra:

- ✓ Reporte de incidencia (Ciudadano)
- ✓ Gestión y asignación de rutas (Administrador)
- ✓ Recepción de ruta (Conductor)

4.2.1.1. DFD proceso crítico de gestion de incidencias y asignación de rutas



4.2.1.2. Descripción del Flujo de Datos de proceso crítico (DFD)

4.2.1.2.1. Entradas

- ☒ Reporte de Incidencia (foto, ubicación, tipo)
- ☒ Datos de contenedores (nivel de llenado, ubicación)
- ☒ Datos de conductores (disponibilidad, vehículo asignado)
- ☒ Parámetros del administrador (prioridad, zona, criterios)

4.2.1.2.2. Transformaciones

- ☒ Validación y registro de Incidencia: según formato, ubicación y se almacena en la BD
- ☒ Evaluación y priorización: según la gravedad, ubicación y tipo
- ☒ Asignación de ruta: Incidencias + contenedores + disponibilidad de conductores
- ☒ Notificación al conductor: sistema envía ruta al dispositivo del conductor

4.2.1.2.3. Salidas

- ☒ Incidencia registrada
- ☒ Incidencia priorizada
- ☒ Ruta generada
- ☒ Notificación enviada al conductor
- ☒ Confirmación de recepción

4.2.2 Diccionario de datos conceptual

4.2.2.1. Tabla de entidades

Entidad	Atributos	Descripción
Usuario	-ID (identificador personal) -Nombre y apellidos -Correo electrónico -Contraseña (encriptada) -Rol del usuario (ciudadano/conductor/administrador)	Es la persona que interactúa con el sistema. Puede ser ciudadano, conductor, administrador.
Ciudadano	-ID (identificador personal) -Nombre y apellidos -Correo electrónico -Contraseña (encriptada) -Rol del usuario (ciudadano/conductor/administrador)	Es el usuario que consulta la información y puede reportar incidencias
Conductor	-Carnet de conducir (datos del carnet de conducir) -Disponibilidad (turno, días) -Teléfono o Tablet (emergencia, notificaciones)	Es el usuario encargado de realizar las rutas de recogida.
Contenedor	- ID (código único) - Tipo de residuo (vidrio, orgánico, plástico, resto) - Coordenadas (latitud, longitud) - Capacidad (litros, kilos) - Nivel de llenado actual (0%- 100%) - Estado operativo (activo, en mantenimiento, averiado)	Recipiente cuya función la recogida de residuos de la vía pública
Camión	-Matrícula (identificador único) -Capacidad de carga (kg o litros) -Tipo de actividad (recogida normal, mantenimiento)	Vehículo utilizado para la recogida de residuos de contenedores
Ruta	-ID (código de ruta) -Fecha y hora o turno -Camión asignado -Conductor asignado -Lista ordenada de contenedores (paradas)	Recorrido predefinido pasando por la lista de contenedores determinados que un conductor debe visitar en ese día.
Incidencia	-ID -Fecha y hora -Contenedor afectado -Tipo de incidencia (contenedor lleno, roto, averiado, otro) -Ciudadano que reporta la incidencia -Foto (opcional) -Estado de la incidencia (pendiente, en proceso, resuelta)	Reporte de realizado por un usuario (ciudadano o conductor) de un problema en un contenedor
Notificación	-ID -Fecha de envío -Destinatario (usuario) -Tipo de mensaje (app, email) -Contenido (texto)	Mensaje enviado desde el sistema a un usuario (ciudadano o conductor)

4.2.2.2. Cardinalidades y justificación

Relación	Cardinalidad	Justificación
Usuario - Ciudadano	Un usuario - un ciudadano	Cada ciudadano es un tipo de usuario
Usuario - Conductor	Un usuario - un conductor	Cada conductor es un tipo de usuario
Usuario - Administrador	Un usuario - un administrador	Cada administrador es un tipo de usuario
Ruta - Contenedor	Una ruta - muchos contenedores	Dentro de una ruta existen varios contenedores Un contenedor puede estar en distintas rutas.
Camión - Ruta	Un camión - muchas rutas	Un camión puede realizar varias rutas (días). Cada ruta utiliza un solo camión en un día.
Conductor - Ruta	Un conductor - muchas rutas	Un conductor puede realizar varias rutas en diferentes fechas Cada ruta es asignada a un único conductor en una fecha determinada.
Ciudadano- Incidencia	Un ciudadano- muchas incidencias	Un ciudadano puede reportar muchas incidencias Cada incidencia pertenece a un único ciudadano.
Contenedor - Incidencia	Un contenedor - muchas incidencias	Un contenedor puede tener varias incidencias en diversas fechas Cada incidencia pertenece solo a un contenedor
Usuario - Notificación	Un usuario - muchas notificaciones	Un usuario puede recibir varias notificaciones. Cada notificación va dirigida a un único usuario.

- ☑ (1:N) -> Camión - Ruta : Un camión puede realizar varias rutas, pero cada ruta pertenece a un único camión.
- ☑ (N:M) -> Ruta - Contenedor: Una ruta incluye varios contenedores y un contenedor puede aparecer en distintas rutas.
- ☑ (1:N) -> Ciudadano - Incidencia: Un ciudadano puede reportar múltiples incidencias

4.3 Modelado orientado a objetos y UML

4.3.1 Diagrama de Casos de Uso

4.3.1.1. Imagen del diagrama y tabla

Actor	Caso de uso	Descripción	Include / Extend
Ciudadano	CU01 Reportar incidencia	Envía incidencia con foto y ubicación	Include: Adjuntar foto
Ciudadano	CU02 Recibir notificación	Recibe aviso de resolución	—
Conductor	CU03 Marcar recogida	Indica que un contenedor fue vaciado	—
Conductor	CU04 Recibir ruta	Recibe ruta optimizada	Include: Consultar contenedor

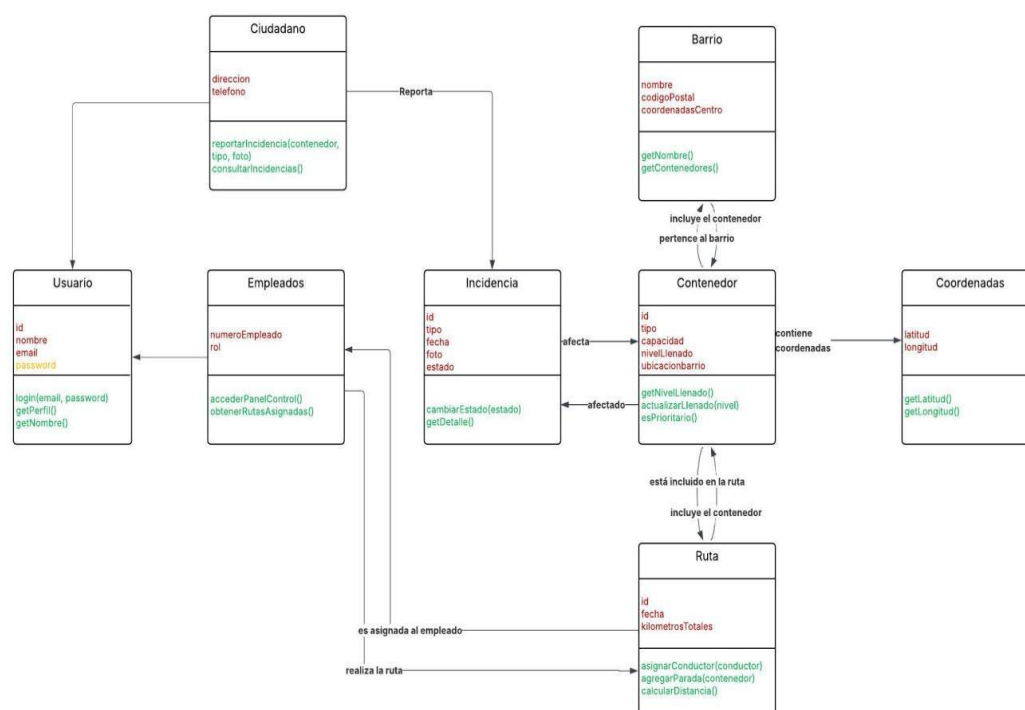
Administrador	CU05 Asignar ruta	Asigna rutas manuales o automáticas	Extend: Notificar conductor
Administrador	CU06 Gestionar conductores	CRUD de conductores	—

4.3.2 Diagrama de Clases del dominio

El diagrama de clases del dominio de Ecologistics es la representación gráfica de los principales conceptos del sistema de negocio de la gestión de residuos (ciudadano, contenedor, ruta, etc.), sus correspondientes atributos (ID, nombre, nivel de llenado, etc.), métodos (iniciar sesion, gestionar contenedores, etc.) y las relaciones que existen entre ellos (herencias, encapsulamiento, etc.).

En el diagrama de clases podemos ver todas esas relaciones.

4.3.2.1. Diagrama conceptual



4.3.2.2. Tabla de Clases

Con el fin de establecer el diagrama de clases del dominio establecemos siguiente tabla de clases según los atributos:

Clase	Atributos	Métodos públicos	Relaciones
Usuario	Id nombre email password	login(email, password) getPerfil() getNombre()	Usuario es la superclase
Ciudadano	direccion telefono	reportarIncidencia (contenedor, tipo, foto) consultarIncidencias()	1:N con Incidencia
Empleado	numero Empleado Rol (conductor/administrado)	accederPanelControl() obtenerRutasAsignadas()	1:N con Ruta

	r)		
Contenedor	Id nivelLlenado (0-100) ubicacion (en coordenadas) barrio	getNivelLlenado() actualizarLlenado(nivel) esPrioritario()	N:M con Ruta
Barrio	nombre codigoPostal coordenadas (long;lat)	getNombre() getContenedores()	1:N con Contenedor
Ruta	id fecha conductor nombreEmpleado paradas totaldeparadas	asignarConductor agregarParada(contenedor)	N:M con Contenedor

Atributos privados: en rojo

Atributos protegidos: en naranja

Atributos públicos: en verde

4.3.3 Diagramas de Secuencia

4.3.3.1. Secuencia 1: Ciudadano reporta incidencia

4.3.3.1.1. Pasos de la Secuencia 1

4.3.1.1.2. Diagrama de la Secuencia 1

4.3.3.2. Secuencia 2: Login conductor y entrega de ruta

4.3.3.2.1. Pasos de la Secuencia 2

4.3.3.2.1. Diagrama de la Secuencia 2

5 DISEÑO TÉCNICO E IMPLEMENTACIÓN

5.1 Selección CASE y entorno de trabajo

5.1.1 Upper CASE

5.1.1.1. Herramienta elegida

☒ Herramienta seleccionada: Lucichart

☒ Justificación:

Entre las dos herramientas analizadas, Lucichart destaca como la opción más adecuada para este proyecto, dado que al ser muy visual e intuitiva con múltiples plantillas que combina tanto la facilidad de uso como de aprendizaje para los trainees. Esta dinámica acelera el aprendizaje al reducir la complejidad y facilitar la explicación y mentoría de los seniors mientras se construyen los diagramas UML o cualquier otro tipo de diagrama necesario. Además se trata de una plataforma online que no requiere instalación, permitiendo al equipo no solamente trabajar en tiempo real sino también simultáneamente en el mismo diagrama, favoreciendo la colaboración desde el inicio de forma segura.

5.1.1.2. Instrucciones de uso

Instrucciones de uso & Plantilla para diagramas

☒ Plantilla UML básica

☒ Título; Autor; Fecha; Versión; Leyenda (símbolos y visibilidad).

☒ Capas recomendadas:

a) Diagrama lógico

b) Diagrama de despliegue

c) Diagrama de clases

d) DFD

☒ Convenciones

Nombres en inglés/español consistentes; atributos privados con -, públicos con + (colores); relaciones con multiplicidad.

☒ Checklist antes de exportar

revisar PK/FK en clases de dominio; comprobar nombres de actores; exportar PNG 300-600 dpi para anexos.

5.1.2 Lower CASE

5.1.2.1 IDE y control de versiones

Herramienta principal

Visual Studio Code como IDE estándar del equipo.

Git + GitHub como sistema de control de versiones y plataforma de colaboración.

a) Lower-CASE seleccionado: Entorno de Desarrollo Integrado - Visual Studio Code

Justificación:

Entorno de Desarrollo Integrado

Entre las herramientas elegimos Visual Studio Code porque es un entorno de desarrollo que facilita mucho el aprendizaje. Su interfaz es sencilla, muestra el código de forma clara y ofrece ayudas visuales (partes del código). El autocompletado y las extensiones son una guía constante que sugieren tanto funciones, como fallos y facilitan entender las estructuras de HTML, CSS, JavaScript; lo que para un trainee significa aprender al mismo tiempo que escribe código. Esta herramienta guía, acompaña y enseña con seguridad.

b) El Control de Versiones

Herramientas seleccionadas: (Git/GitHub)

Justificación:

Elegimos Git + GitHub porque son herramientas fundamentales para aprender a trabajar como desarrollador desde el primer día.

Git por sí sólo permite practicar con comandos básicos de control de versiones y ayuda a los trainees a entender cómo guardar los cambios y volver atrás si cualquier cosa falla y de esta forma poder ver la evolución del proyecto.

Por su parte GitHub, contribuye facilitando el trabajo en equipo al compartir el código, permite revisar las aportaciones hechas por los compañeros así como colaborar ordenadamente.

Para los trainees un gran beneficio ya que pueden experimentar sin miedo a perder código, y aprender en un entorno real y profesional.

c) Herramientas De Testing Automatizado seleccionadas Jest + Supertest & Cypress - Pruebas End-to-End (E2E)

i. Jest + Supertest - Pruebas de API y unidad

Elegimos Jest y Supertest porque encajan con el backend en Node.js.

Jest es una herramienta ampliamente usada tanto frontend como backend; y Supertest permite comprobar con facilidad la respuesta de los endpoints de los APIs.

Los tests se realizan rapidamente y por ello el código puede ser validados antes de subirlo. Para los trainees el aprendizaje es fácil y les permite realizar pruebas simples. Además es gratuito.

ii. Cypress - Pruebas End-to-End (E2E)

Elegimos Cypress porque permite probar la aplicación como si fuera un usuario real tanto abriendo un navegador, como con escritura y navegación. Esto nos permite simular y validar los flujos del ciudadano y también del conductor.

Cypress tiene amplios beneficios para los trainees porque muestra tanto videos como capturas de pantalla y permite entender lo que ha ocurrido. Su interfaz en tiempo real hace que escribir tests sea más visual y menos intimidante. Es gratuito y dispone de una gran comunidad. La herramienta detecta errores que las prueba de API no pueden ver y además la aplicación funciona de principio a fin con botones, formularios, mensajes, etc.

Cobertura: publicar informe de cobertura en CI; objetivo inicial 80% (ajustable).

5.1.2.2. Seguridad y carga

Herramientas Principales:

DAST: OWASP ZAP para escaneos de seguridad web.

Pruebas de carga: k6 para scripts de carga y definición de umbrales.

a) OWASP ZAP - Pruebas de seguridad

Añadimos a las anteriores herramientas OWASP ZAP con el fin de añadir desde el inicio una herramienta de seguridad y protección de datos. Esta herramienta detecta automáticamente cualquier vulnerabilidad web. Su modo automático es ideal para trainees ya que permite un escaneo completo en pocos clics sin ser expertos en ciberseguridad. Además, se integra con facilidad con CI/CD analizando cada cambio en el código. Es una herramienta de código abierto que facilita cumplir con el Reglamento General de Protección de Datos (RGPD) evitando así que puedan filtrarse datos sensibles de los ciudadanos que accedan a la plataforma y protegiendo la aplicación frente a cualquier primer ataque.

b) k6 - Pruebas de carga y rendimiento

Elegimos k6 porque permite simular cientos o miles de usuarios a la vez, algo que es necesario dado que Ecologistics Web debe soportar “miles de ciudadanos conectados simultáneamente”.

Es una herramienta ligera, que puede ejecutarse en el pipeline sin ralentizar el sistema. Además, usa también JavaScript facilitando al equipo completo que pueda escribir scripts sin tener que aprender un nuevo lenguaje. Es una aplicación gratuita que permite definir umbrales de rendimiento. Esta herramienta garantiza la buena experiencia del ciudadano y de los empleados dado que asegura el funcionamiento de la plataforma Ecologistics al evitar las caídas y la lentitud del sistema causadas por la alta demanda.

5.1.2.2.1. - Integración operativa

☒ SAST: SonarCloud (análisis estático) en CI.

- ☑ DAST en pipeline: ejecutar OWASP ZAP contra entorno de staging tras despliegue; fallar pipeline si se detectan vulnerabilidades críticas.
- ☑ Carga en CI: ejecutar k6 como smoke test en staging; definir umbrales (ej.: 95% de requests < 500 ms; error rate < 1%).
- ☑ Gestión de secretos: almacenar credenciales y tokens en GitHub Secrets; no incluir secretos en repositorio.

5.1.2.2.2. Buenas prácticas de seguridad

- ☑ Autenticación: JWT con expiración corta y refresh tokens controlados.
- ☑ Transporte: TLS obligatorio en todos los endpoints.
- ☑ Almacenamiento: hashing de contraseñas con bcrypt; cifrado en reposo para datos sensibles si aplica.
- ☑ Registro y auditoría: logs estructurados (JSON) con trazabilidad de eventos de seguridad; política de retención documentada.

5.1.3. I-CASE e integración CI/CD

5.1.3.1. Objetivo

Definir y documentar los pipelines de integración y entrega continua que automatizan linting, testing, análisis estático y dinámico de seguridad, pruebas de rendimiento básicas, construcción de artefactos y despliegue a entornos controlados. Los pipelines deben ser reproducibles, auditable y bloquear merges o releases cuando fallen los quality gates definidos.

5.1.3.2. Herramienta seleccionada GitHub Actions

Herramienta seleccionada – GitHub

Herramienta seleccionada para CI/CD: GitHub Actions

Justificación:

Elegimos GitHub Actions porque permite que las pruebas se ejecuten automáticamente cada vez que alguien sube código al repositorio, lo que garantiza que la calidad sea revisada de forma continua sin la implicación de los trainees. Al recibir feedback inmediato pueden ver al instante si algo falla, entender qué ha pasado y corregirlo al instante. Todo esto reduce errores, ayuda a adoptar buenas prácticas y aumenta su curva de aprendizaje. GitHub Actions es una herramienta que acompaña y facilita el aprendizaje de los trainees.

5.1.3.3. Flujo recomendado del pipeline

Definir en el repositorio un workflow reproducible que incluya, como mínimo, los siguientes jobs en secuencia o con dependencias:

- ✓ lint (ESLint/Prettier) – comprobar estilo y errores estáticos.
- ✓ test (Jest + Supertest) – ejecutar unit/integration tests y generar cobertura.
- ✓ sonar (SonarCloud) – análisis estático y aplicación del quality gate.
- ✓ security-scan (OWASP ZAP) – DAST contra staging; bloquear si hay vulnerabilidades críticas.
- ✓ performance-smoke (k6) – pruebas de humo en staging con umbrales.
- ✓ build – construir artefacto Docker y etiquetado.
- ✓ publish/deploy – publicar imagen y desplegar a staging; despliegue a producción con aprobación.

5.1.3.3. Quality gates y umbrales

SonarCloud: Coverage mínimo sugerido: 80%; no new critical/blocker issues en PRs.

DAST (OWASP ZAP): bloquear merge si se detectan vulnerabilidades critical/high sin mitigación documentada.

Performance (k6): umbrales iniciales sugeridos: 95% requests < 500 ms; error rate < 1%; en PRs marcar como warning y bloquear despliegue si se superan en staging.

5.1.3.4. Gestión de secretos y reporting

- ✓ Dónde guardar secretos: GitHub Secrets (ej.: SONAR_TOKEN, DOCKER_REGISTRY_USER, DOCKER_REGISTRY_PASS, ZAP_API_KEY, K6_API_KEY).
- ✓ Reporting: publicar artefactos de cobertura, reportes Sonar y ZAP como artefactos del workflow y enlaces en la página del PR.

5.2 Arquitectura del software y despliegue

5.2.1 Arquitectura lógica y patrón

5.2.1.1. Patrón Arquitectónico: MVC / API REST.

a. Descripción

El sistema adopta el patrón Modelo-Vista-Controlador (MVC) combinado con una API REST que expone los recursos del dominio para clientes web y móviles. Esta separación facilita mantenibilidad, pruebas y escalado.

b. Responsabilidades por capa

☒ Capa Presentación (Vista)

SPA (React u otro) que consume la API; validación básica en cliente.

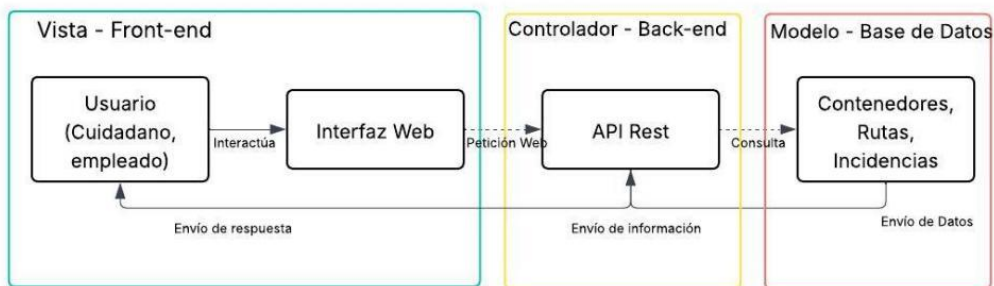
☒ Capa Control (API)

Controladores por recurso, orquestación de lógica de negocio, autenticación/autorizar, manejo centralizado de errores.

☒ Capa Modelo (Dominio / Persistencia)

Entidades del dominio, repositorios/DAOs, migraciones y validaciones de integridad.

5.2.1.2. Diagrama lógico



5.2.2 Arquitectura de despliegue

5.2.2.1. Diagrama de despliegue navegador → CDN → API → BBDD → cache.

Descripción

El despliegue contempla clientes accediendo a recursos estáticos servidos por CDN, peticiones dinámicas dirigidas a la API desplegada en contenedores, persistencia en PostgreSQL y aceleración/almacenamiento temporal en Redis. El despliegue puede orquestarse en Kubernetes o en una PaaS según la estrategia operativa

5.2.2.2. Componentes Principales

- ☒ PostgreSQL (persistencia relacional)
- ☒ Redis (cache y colas ligeras si aplica)
- ☒ Docker (contenedorización de servicios)
- ☒ Kubernetes / PaaS (orquestación y hosting)

5.2.2.2. Diagrama de Despliegue

5.2.3 ADR Elección de Base de Datos

5.2.3.1. ADR completo

Título: Elección de Base de Datos Relacional para el Sistema de Gestión de Residuos EcoLogistics (Ciudadanos, Incidencias, Rutas y Contenedores)

1. Contexto

EcoLogistics Web es una plataforma para la gestión de residuos urbanos con tres ámbitos principales: comunicación de incidencias por ciudadanos, optimización de rutas para conductores y administración de contenedores y operaciones por el personal gestor.

La información crítica controlada por el sistema es:

- ☒ Ciudadanos: datos personales, reportes de incidencias
- ☒ Contenedores: ubicación, tipo, nivel de llenado, estado operativo
- ☒ Incidencias: reportes (contenedores llenos, rotos, etc.)
- ☒ Rutas logísticas: asignación de conductores, orden de paradas, tiempos
- ☒ Conductores y vehículos: disponibilidad, turnos, capacidad

Requisitos relevantes: integridad referencial, consistencia transaccional en operaciones críticas, consultas relacionales y geoespaciales, auditoría y cumplimiento RGPD.

2. Decisión

Se selecciona una base de datos relacional (RDBMS) como tecnología principal para la persistencia de la información para la plataforma EcoLogistics Web. La implementación recomendada es PostgreSQL (v15+) con PostGIS habilitado para soporte geoespacial. PostgreSQL será la tecnología oficial desde el inicio del proyecto.

3. Alternativas Consideradas

- ☒ Base de datos NoSQL (MongoDB)

Pros: esquema flexible, escalado horizontal sencillo

Contras: integridad referencial limitada, menos idónea para joins complejos y auditoría

- ☒ Base de datos relacional (PostgreSQL, MySQL, SQL Server)

Pros: relaciones estructuradas, transacciones ACID, integridad referencial, SQL robusto; PostgreSQL aporta además capacidades geoespaciales (PostGIS).

Contras: escalado horizontal más complejo que en algunas NoSQL; requiere planificación de esquema y migraciones.

Decisión tomada en favor de la opción relacional por su ajuste a los requisitos funcionales y no funcionales del proyecto.

4. Justificación

La elección del modelo relacional se fundamenta en los siguientes motivos:

- ☒ Las entidades del dominio presentan relaciones fuertes y bien definidas (un ciudadano reporta incidencias, un camión recoge múltiples contenedores, cada incidencia pertenece a un contenedor concreto).
- ☒ Las operaciones clave del negocio requieren consistencia estricta: creación de incidencias, actualización del nivel de llenado, asignación de rutas.
- ☒ El sistema ejecutará consultas complejas y frecuentes, incluyendo joins entre incidencias, contenedores, barrios y consultas geoespaciales.
- ☒ La auditoría (quién hizo qué y cuándo) y el cumplimiento RGPD exigen trazabilidad y control de integridad
- ☒ PostgreSQL + PostGIS cubre las necesidades relacionales y geoespaciales, dispone de un ecosistema maduro y se integra bien con Kubernetes/PaaS y herramientas de migración.

5. Consecuencias

Positivas

- ☒ Se garantiza la integridad referencial entre entidades críticas
- ☒ Las transacciones ACID permiten mantener consistencia fuerte en operaciones sensibles.
- ☒ SQL facilita la ejecución de consultas complejas y optimizadas.

- ☑ La auditoría y trazabilidad se implementan de forma más directa
- ☑ La Compatibilidad RGPD facilita el control y el borrado/portabilidad con procedimientos definidos

Negativas

- ☑ La escalabilidad horizontal es más limitada; puede requerir réplicas de lectura, particionado o sharding en fases avanzadas.
- ☑ Mayor planificación de esquema y coste de cambios estructurales (migraciones controladas).
- ☑ Coste operativo asociado a alta disponibilidad, backups y monitorización.

6. Información operativa mínima

Criterios de aceptación (mínimos)

- ☑ 95% de consultas críticas con latencia < 200 ms en condiciones normales
- ☑ Soportar 200 conexiones concurrentes con rendimiento aceptable
- ☑ RTO ≤ 1 hora; RPO ≤ 15 minutos para datos críticos

Impacto operativo y medidas mínimas

- ☑ Backups: incrementales diarios y full semanal; retención 30 días.
- ☑ Alta disponibilidad: despliegue con réplicas de lectura y mecanismo de failover (servicio gestionado o Patroni/PGPool)
- ☑ Seguridad: TLS en tránsito; cifrado en reposo según proveedor; control de accesos por roles; logging de auditoría
- ☑ Monitorización: métricas básicas (latencia, conexiones, replicación, uso disco) y alertas configuradas

Plan de migración / rollback (resumen)

- ☑ Flujo: migraciones en staging -> pruebas de integración-> snapshot/backup previo -> despliegue en producción
- ☑ Rollback: restauración desde backup o ejecución de scripts de reversión según el cambio

Pruebas mínimas / PoC a ejecutar antes de producción

- ☑ PoC con dataset representativo y consultas típicas (incluyendo geoespaciales)
- ☑ Pruebas de carga básicas (k6) para validar latencias y concurrencia
- ☑ Validación de failover y recuperación desde backup

Coste y operación

- ☑ Evaluar coste estimado entre servicio gestionado (PaaS) y autogestión; documentar en anexos la opción elegida y su coste mensual aproximado

6. Estado

El estado es Aprobado.

La base de datos relacional (PostgreSQL + PostGIS) será utilizada como tecnología oficial desde el inicio del proyecto.

5.3 Diseño de datos físico y APIs

Se define el modelo físico de datos, el diagrama ER, los contratos API y el flujo de autenticación. Los artefactos referenciados deben añadirse al repositorio en las rutas indicadas.

5.3.1 Modelo relacional físico

Artefactos y rutas

- ☑ Esquema físico (DDL) : /docs/schema/physical_schema.sql
- ☑ Diagrama ER: /docs/diagrams/05_physical_model.svg

Notas de diseño

- ✓ geom usa PostGIS; si no está disponible, usar latitude/longitude (DECIMAL).

- ✓ stops puede ser JSONB para simplicidad; si se requiere orden y consultas relacionales, usar route_stops.
- ✓ Índices: GIST en geom; B-tree en código, matrícula, estado.
- ✓ Auditoría: considerar triggers o tabla audit_log para cambios críticos.

☒ Product Backlog / Historias de Usuario

Historias de Usuario

- ✓ US01 - Como ciudadano, quiero comunicar una incidencia (como un contenedor dañado o basura acumulada) adjuntando foto y ubicación, para que el ayuntamiento pueda gestionarla con rapidez.
- ✓ US02 - Como ciudadano, quiero recibir un aviso cuando la incidencia que he enviado haya sido resuelta, para confirmar que la actuación se ha llevado a cabo.
- ✓ US03- Como conductor, quiero poder indicar que un contenedor ha sido vaciado, para que el sistema registre su nivel de llenado como 0%.
- ✓ US04 - Como conductor, quiero recibir en mi móvil el itinerario optimizado de la jornada, para reducir el consumo de combustible y aprovechar mejor el tiempo.
- ✓ US05 - Como administrador, quiero asignar manualmente una ruta adicional a un camión cuando se produzca un aumento inesperado de residuos, para evitar que los contenedores se saturen.
- ✓ US06 - Como administrador, quiero gestionar los perfiles de los conductores (crear, eliminar o actualizar), para mantener la información del equipo siempre al día.

El propósito de este documento es entregar un Plan de Dirección que permita pasar de la programación a la Ingeniería y producir el MVP operativo de la plataforma Ecologistics Web.

5.3.2 Contratos API y ejemplos

Artefactos y rutas

- ✓ Contrato OpenAPI: /docs/api/openapi.yaml
- ✓ Resumen legible con ejemplos: /docs/api/contract.md
- ✓ Flujo de autenticación y headers: /docs/api/auth.md

5.3.2.1. Listado de endpoints

método, ruta, descripción, request example, response example, códigos HTTP.

5.3.2.2. Autenticación

JWT flow y headers.

5.4 Lógica crítica y algoritmos

Objetivo

Entregar pseudocódigo mínimo (estructurado y heurístico), parámetros y casos de prueba para validar el algoritmo de asignación de rutas.

5.4.1 Algoritmo de asignación de rutas

Descripción

El algoritmo selecciona contenedores candidatos por nivel de llenado y restricciones, agrupa por zona, asigna paradas a camiones según capacidad y ventanas, y aplica optimización local (2-opt) para reducir distancia.

5.4.1.1.Pseudocódigo

5.4.1.1.1. Versión estructurada

```

FUNCION generar_rutas(containers, camiones, umbral, capacidad_camion):
  candidatos = FILTER(containers, c -> c.nivel_llenado >= umbral AND c.estado = 'activo')
  zonas = AGRUPAR_POR_ZONA(candidatos)
  rutas = []

```

```

PARA cada zona EN zonas:
  lista = ORDENAR_POR_PRIORIDAD_Y_DISTANCIA(candidatos_de_zona)
  MIENTRAS lista NO VACIA:
    ruta = NUEVA_RUTA()
    capacidad_restante = capacidad_camion
    MIENTRAS lista NO VACIA Y capacidad_restante > 0:
      cont = POP_PRIMERA(lista)
      SI cont.volumen_estimado <= capacidad_restante:
        AÑADIR_PARADA(ruta, cont)
        capacidad_restante -= cont.volumen_estimado
      APLICAR_2OPT(ruta.paradas)
    AÑADIR rutas, ruta
  RETURN rutas

```

5.4.1.1.2. Versión estructurada

1. Seleccionar contenedores con nivel $\geq$ UMBRAL.
2. Para cada camión: elegir siguiente contenedor más cercano con mayor nivel que quepa.
3. Repetir hasta cubrir capacidad; aplicar 2-opt por ruta.
4. Si quedan contenedores, crear rutas adicionales o marcar intervención manual.

5.4.1.2. Parámetros y restricciones

Parámetro	Unidad	Valor por defecto	Comentario
CAPACIDAD_CAMION	litros	8000	Capacidad útil por camión
UMBRAL_LLENADO	%	70	Candidatos si $\geq$ umbral
VENTANA_INICIO	hh:mm	0.333333333	Inicio ventana servicio
VENTANA_FIN	hh:mm	0.75	Fin ventana servicio
TIEMPO_MAX_CALCULO	s	30	Tiempo máximo por lote
PRIORIDAD	enum	alta, media, baja	Prioridad de incidencias/contenedores

Usar estructuras de Selección (SI nivel > 70% ENTONCES recoger)

5.4.2 Casos de prueba y validación del algoritmo “Asignación de Rutas”

5.4.2.1. Tabla de casos de prueba

Para garantizar que el motor de optimización de rutas se comporta de manera predecible y robusta ante escenarios reales, se definen los siguientes casos de prueba unitarios y de integración.

Estos casos validan tanto la lógica estructurada del algoritmo (Secuencia, Selección e Iteración) como la interacción con la extensión geoespacial ****PostGIS****

ID Caso	Descripción del Escenario	Datos de Entrada	Resultado Esperado
TC-01	Ruta básica (Filtro >70%)	10 contenedores, 1 camión (8000 L). 4 contenedores con nivel $\geq$ 70%.	Se genera una única ruta que incluye estrictamente los 4 contenedores con nivel $\geq$ 70%.

TC-02	Capacidad de camión insuficiente	50 contenedores con nivel > 70%, 1 camión (8000 L).	El sistema genera múltiples rutas o sugiere asignación de más vehículos; ninguna ruta excede la capacidad del camión.
TC-03	Ventanas horarias solapadas	Contenedores con restricciones horarias de acceso.	Las rutas respetan los intervalos permitidos; los contenedores fuera de ventana quedan sin asignar.
TC-04	Priorización de contenedores críticos	Mezcla de contenedores con y sin prioridad alta.	Los contenedores de alta prioridad aparecen al principio de la ruta optimizada.
TC-05	Optimización de dispersión geográfica	Contenedores muy dispersos geográficamente (distancia > 5 km).	La heurística 2-opt reduce significativamente la distancia total respecto a la ruta inicial.
TC-06	Tolerancia a fallo de camión	Un camión asignado pasa a estado en taller antes de iniciar la ruta.	El sistema reasigna la ruta a otro camión disponible o la marca para intervención manual.

5.4.2.2. Estrategia de validación

- ☑ Unit tests: Jest para funciones de selección y cálculo.
- ☑ Integración: Supertest para endpoints que generan rutas
- ☑ E2E: Cypress para flujos conductor/ciudadano.
- ☑ Rendimiento: k6; métrica de aceptación: p95 tiempo de cálculo ≤ 30 s para lote de 500 contenedores.
- ☑ Seguridad: OWASP ZAP en staging; bloquear PRs con vulnerabilidades críticas.

US03- Como conductor, quiero poder indicar que un contenedor ha sido vaciado, para que el sistema registre su nivel de llenado como 0%.

5.5 UI UX y accesibilidad

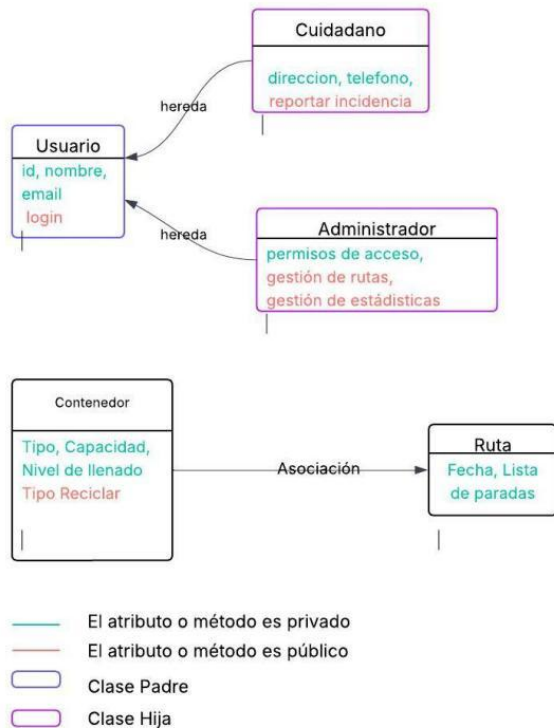
5.5.1 Mockups y prototipos

5.5.1.1. Pantallas

5.5.1.1.1. Archivos y rutas

- ✓ Inicio ciudadano: /docs/ui/home_ciudadano.png
- ✓ Formulario incidencia: /docs/ui/form_incidencia.png
- ✓ Vista conductor: /docs/ui/vista_conductor.png
- ✓ Panel administrador: /docs/ui/panel_admin.png
- ✓ Descripción y rutas: /docs/ui/README.md

5.5.1.1.1 Diagrama de Clases UML y Descripción



Descripción del Diagrama

El daigrama de clases UML contiene las siguientes Clases y sus correspondientes atributos:

♦ Clase Usuario (clase padre)

Atributos:

- Ⓣ ID identifica de forma única al usuario, privado
- Ⓣ Nombre es el nombre completo de usuario, privado
- Ⓣ Email es la dirección de correo electrónico, privado
- Ⓣ Login son las credenciales que dan acceso al sistema si están correctas, público

♦ Clase Ciudadano (clase hija que hereda de Usuario)

Atributos:

- Ⓣ Dirección es la calle, número, código postal y ciudad del ciudadano, privado
- Ⓣ Teléfono es el número de contacto del ciudadano, privado
- Ⓣ Reportar incidencia es el atributo que permite al ciudadano enviar una incidencia sobre un contenedor, público

♦ Clase Administrador (clase hija que hereda de Usuario)

Atributos:

- Ⓣ Permisos de acceso es el número que indica el nivel de permiso que tiene el administrador, privado
- Ⓣ Gestión de la ruta es el atributo que permite al administrador crear, modificar o eliminar las rutas de los camiones o conductores, privado
- Ⓣ Gestión de las estadísticas es el atributo que permite crear los informes de los contenedores, las rutas, las incidencias, público

♦ Clase Contenedor

Atributos:

- Ⓣ Tipo es el atributo que indica cual es el material que recogido por el contenedor, privado
- Ⓣ Capacidad es la cantidad máxima en litros que puede almacenar el contenedor, privado

⬇ Nivel de Llenado es la cantidad que indica el porcentaje (de 0 a 100) de llenado del contenedor, público

⬇ Tipo de reciclado es el atributo que indica como será reciclado el material que se encuentra al interior del contenedor que ha sido recogido.

Este tipo de atributo se considera como un método polímorfico ya que cambia según el tipo de contenedor porque cada material se trata de una manera distinta.

- Si el contenedor es de vidrio, el proceso que el material es distinto ya que hay que calentarlo hasta que se funde y luego darle forma).

- Si el contenedor es de papel, el proceso que el material es diferente ya que aplastarlo, triturarlo y convertirlo en pasta para luego luego volver a fabricar papel de nuevo.

Clase Ruta

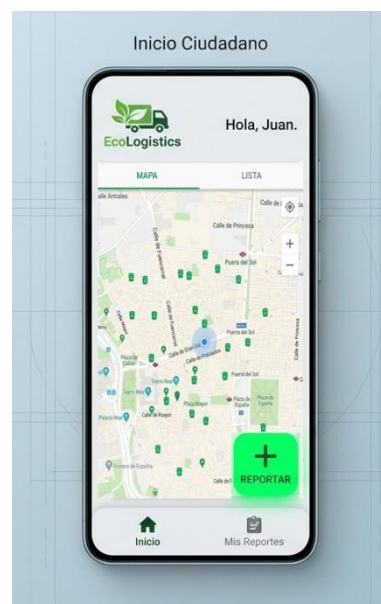
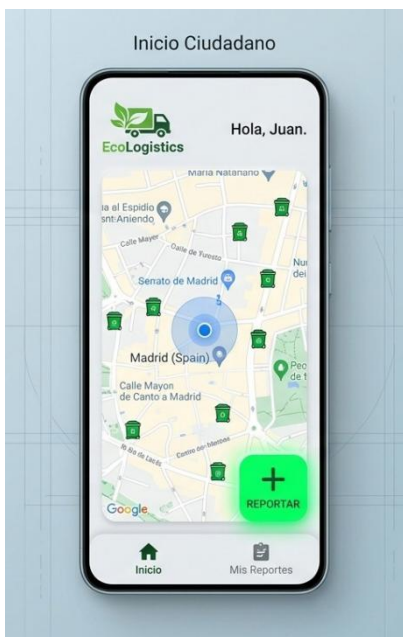
Atributos:

⬇ Fecha de la Ruta corresponde a la fecha en la que se efectúa la ruta de recogida, privado

⬇ Lista de Paradas corresponde a una lista que incluye los contenedores que será visitados por el camión y conductor que realiza la ruta, privado

5.5.1.1.2. Descripción de pantallas

- ✓ Inicio ciudadano: botón “Reportar incidencia”, listado de mis incidencias
- ✓ Formulario incidencia: foto, ubicación (mapa), tipo, descripción, consentimiento RGPD.
- ✓ Vista conductor: lista de rutas del día, mapa con paradas, botón “Marcar contenedor vaciado”.



Figuras 1 y 2 - Pantalla de inicio del ciudadano

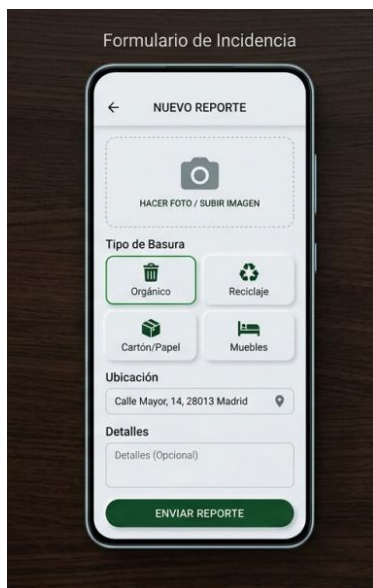


Figura 3 - Formulario de incidencia (Ciudadano)

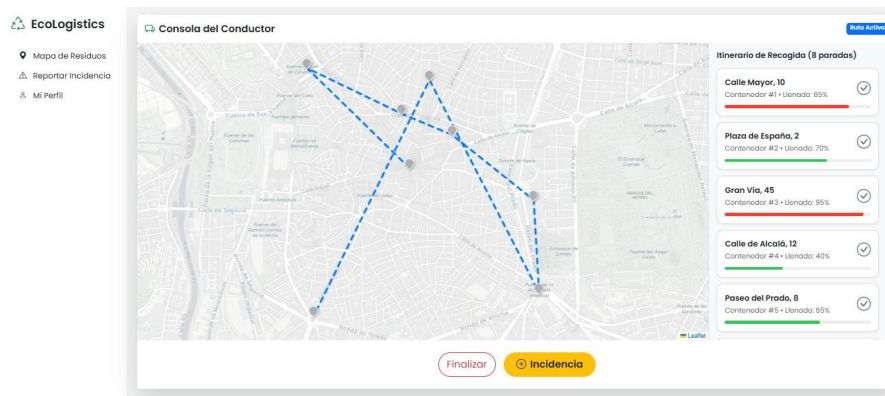


Figura 4 - Vista del Panel de Conductor (Sin llegada a Parada)

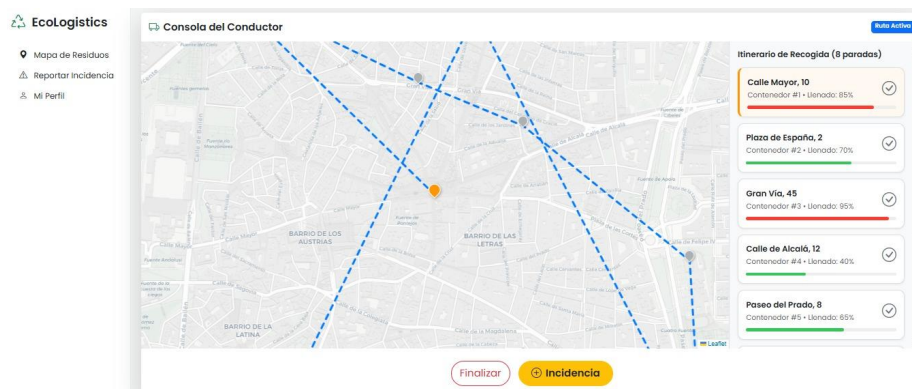


Figura 5 - Vista del Panel de Conductor (En Parada)

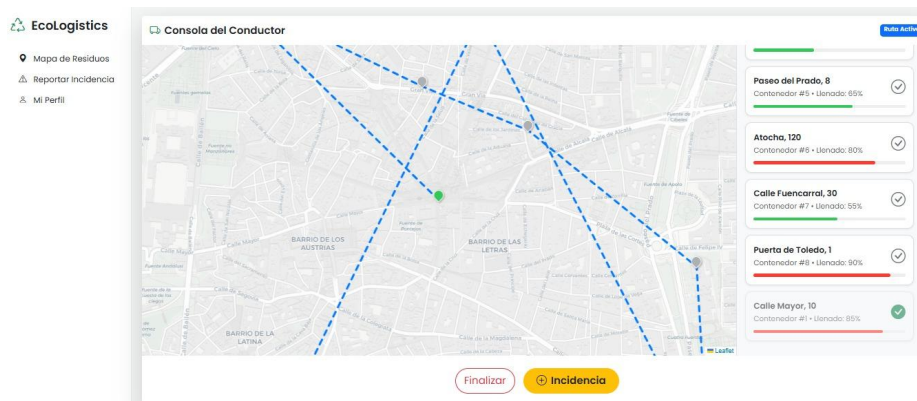


Figura 6 - Vista del Panel de Conductor (Parada Realizada)

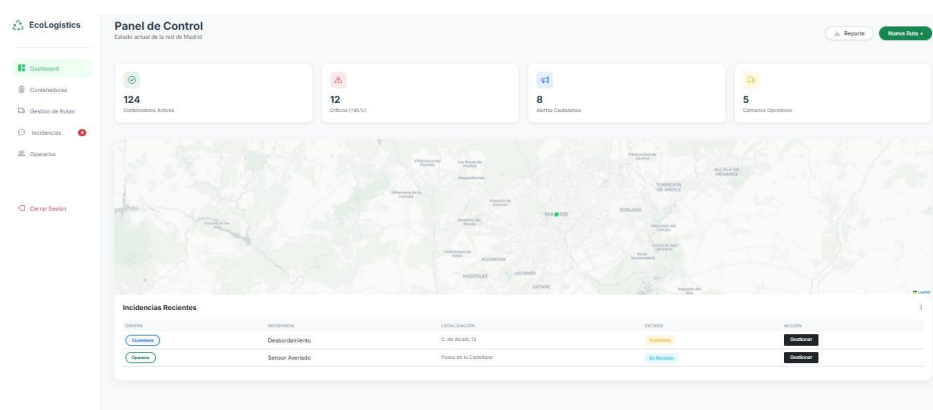


Figura 7 - Vista del Panel de Control del Administrador

5.5.2 Reglas de diseño y criterios WCAG

5.5.2.1. Nivel objetivo y reglas.

- ☒ Nivel objetivo: WCAG 2.1 AA
- ☒ Reglas:
 - ✓ Contraste $\geq 4.5:1$
 - ✓ Tamaño táctil $\geq 44px$
 - ✓ Navegación por teclado
 - ✓ Textos alternativos en imágenes

5.5.2.2. Criterios de aceptación de UI

- ✓ flujos clave ≤ 3 interacciones
- ✓ mensajes de error claros y localizables

5.6 Seguridad y cumplimiento

5.6.1 Protección de datos y RGPD

Herramientas De Testing Automatizado seleccionadas:

Jest + Supertest & Cypress - Pruebas End-to-End (E2E) & k6 & OWASP ZAP.

5.6.1.1. Políticas

- ✓ Minimización: recoger solo datos necesarios (email, nombre, teléfono).
- ✓ Consentimiento: checkbox explícito en formulario de incidencia.
- ✓ Retención: datos personales 2 años; logs 90 días.
- ✓ Responsable del tratamiento: Angelina Caballero

5.6.1.2. Medidas técnicas

- ✓ Hashing con bcrypt
- ✓ TLS obligatorio
- ✓ Cifrado en reposo si procede

5.6.2 Controles en pipeline y auditoría

5.6.2.1. SAST/DAST

- ☑ Integrar SonarCloud (SAST) y OWASP ZAP (DAST) en CI
- ☑ Bloquear PRs con vulnerabilidades de severidad alta/critica.

SonarCloud

Análisis continuo de calidad y seguridad del código

- Integración nativa con GitHub
- definen umbrales objetivos de calidad
- Es el estándar en la industria
- Ayuda 24h: detecta problemas y explica cómo corregirlos
- Los trainees reciben comentarios directamente en el PR y pueden corregirlos

5.6.2.2. Registro de accesos

- ☑ Formato de logs: timestamp | user_id | action | resource | ip | resultado
- ☑ Retención 90 días.
- ☑ Guardar en /ops/logs/ o servicio centralizado.

5.7. Infraestructura y operaciones

5.7.1 Estrategia de despliegue y entornos

5.7.1.1. Entornos

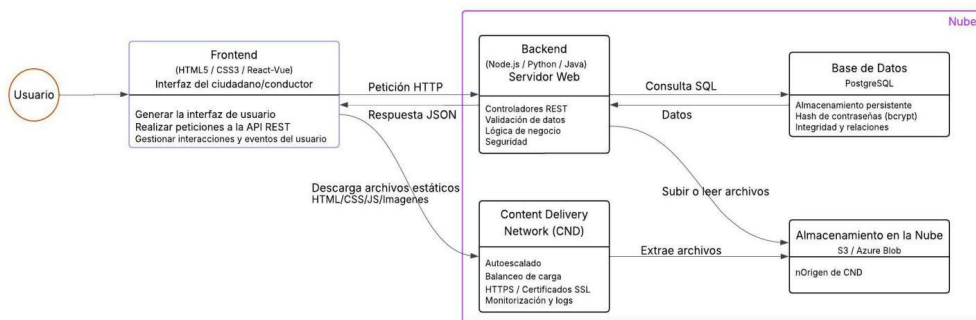
- ☑ Entornos: dev, staging, prod.
- ☑ Sizing recomendado:
 - ✓ dev: 1 vCPU, 2 GB RAM, 20 GB disco.
 - ✓ staging: 2 vCPU, 4 GB RAM, 50 GB disco.
 - ✓ prod: 2 réplicas; cada réplica 2 vCPU, 8 GB RAM, 100 GB disco
 - ✓ BBDD con réplica de lectura.

5.7.1.2. Pipelines CI/CD

CI: build → lint → unit tests → SonarCloud → DAST (ZAP)

CD: deploy staging → smoke tests → deploy prod (aprobación manual).

5.7.1.1.1. Diagrama de Arquitectura Web (navegador - servidor - BBDD)



5.7.2 Monitorización y runbooks

5.7.2.1. Métricas & Runbooks

- ☑ Métricas
 - ✓ latencia API p95
 - ✓ tasa de errores 5xx
 - ✓ CPU/ memoria: Uso disco
 - ✓ Replicación DB lag

5.7.2.2. Alertas y playbooks

Son los pasos de respuesta ante incidentes

- ☑ Runbooks
 - ✓ pasos para diagnóstico y recuperación: runbook_api_failure.md
 - ✓ pasos para failover y restauración: runbook_db_failover.md

6 PLANIFICACIÓN, CALIDAD, RIESGOS Y CIERRE

6.1 Estructura de Desglose del Trabajo EDT WBS

La Estructura de Desglose del Trabajo (EDT / WBS) organiza los entregables y tareas del proyecto en niveles jerárquicos, asigna responsables y facilita la estimación y el control del progreso. La WBS que se presenta a continuación cubre las fases de análisis, diseño, implementación, pruebas, despliegue y cierre.

6.1.1.WBS - Tabla jerárquica

6.1.1.1 Tabla Jerárquica

ID	Entregable / Tarea	Nivel	Responsable
1	Proyecto EcoLogistics Web (MVP)	0	PM
1.1	Gestión del proyecto	1	PM
1.1.1	Planificación y seguimiento	2	PM
1.1.2	Reuniones y comunicación	2	PM
1.2	Análisis	1	Lead Analyst
1.2.1	Product Backlog y RF/RNF	2	BA
1.2.2	Diagramas UML y DFD	2	Analyst
1.3	Diseño	1	Lead Architect
1.3.1	Modelo de datos físico	2	DBA
1.3.2	Contratos API (OpenAPI)	2	Backend Lead
1.4	Implementación	1	Dev Leads
1.4.1	Backend (API)	2	Backend Team
1.4.2	Frontend (UI)	2	Frontend Team
1.4.3	Integración BBDD y PostGIS	2	DBA
1.5	Pruebas y QA	1	QA Lead
1.5.1	Unitarias	2	Devs
1.5.2	Integración	2	QA
1.5.3	E2E y rendimiento	2	QA/DevOps
1.6	Despliegue y Operaciones	1	DevOps
1.6.1	Pipelines CI/CD	2	DevOps
1.6.2	Monitorización y runbooks	2	DevOps
1.7	Entrega y cierre	1	PM
1.7.1	Documentación final y manuales	2	Docs Team
1.7.2	Acta de aceptación y lecciones	2	PM

6.1.2. Estimaciones

6.1.2.1 Tabla de Estimaciones por horas por tarea y perfil

ID	Tarea	Perfil	Horas estimadas
1.1.1	Planificación y seguimiento	PM	16
1.2.1	Product Backlog y RF/RNF	BA	24
1.2.2	Diagramas UML y DFD	Analyst	20
1.3.1	Modelo de datos físico	DBA	12
1.3.2	Contratos API	Backend Lead	12
1.4.1	Backend (endpoints esenciales)	Backend (3 devs)	60
1.4.2	Frontend (4 pantallas)	Frontend (3 devs)	60
1.4.3	Integración BBDD/PostGIS	DBA + Backend	20
1.5.1	Unit tests	Devs	20
1.5.2	Integración tests	QA	16
1.5.3	E2E + rendimiento	QA/DevOps	24
1.6.1	Pipelines CI/CD	DevOps	12
1.6.2	Monitorización & runbooks	DevOps	12
1.7.1	Documentación y manuales	Docs Team	16
1.7.2	Acta de aceptación y cierre	PM	8
Total			337 h

6.2 Cronograma y Diagrama de Gantt

El cronograma organiza las tareas en el tiempo y facilita la identificación de dependencias críticas. A continuación se incluye el Gantt del primer mes y la lista de dependencias de inicio inmediato.

6.2.1. Diagrama de Gantt del primer mes



6.2.2.Tabla del primer mes (con tareas, inicio, duración, y responsable)

Tarea	Inicio	Duración (días)	Responsable
Planificación y kickoff	28/01/2026	3	PM
Product Backlog y RF/RNF	31/01/2026	7	BA
Diagramas UML y DFD	08/02/2026	6	Analyst
Modelo de datos físico (DDL)	15/02/2026	4	DBA
OpenAPI y contratos	19/02/2026	4	Backend Lead
Implementación Backend inicial	23/02/2026	10	Backend Team
Implementación Frontend inicial	23/02/2026	10	Frontend Team
Configuración CI/CD y staging	05/03/2026	5	DevOps
Pruebas unitarias e integración	10/03/2026	7	QA/Devs

6.2.2.Dependencias críticas

6.2.2.1. Lista de FI con justificación

- ☑ FI: la implementación del backend depende del modelo de datos estable para evitar retrabajo en esquemas y migraciones. (Modelo de datos físico)
Justificación: cambios tardíos en el esquema afectan consultas y endpoints.
- ☑ FI: los endpoints deben definirse antes de la implementación para garantizar trazabilidad y pruebas contractuales. Justificación: facilita generación de mocks y pruebas automáticas. (OpenAPI → Backend)
- ☑ FI: pipelines deben estar operativos antes de ejecutar pruebas E2E y DAST en staging. Justificación: automatización garantiza repetibilidad de pruebas. (CI/CD → E2E/DAST)

6.3 Hitos y fechas límite

Se definen hitos clave que marcan entregas parciales y la aceptación final del MVP. Las fechas se fijan para coordinar despliegues, pruebas y validación por parte del cliente.

6.3.1.Hito 1

Backend y BBDD operativos con auditoría de seguridad

Fecha: 19/03/2026

Responsable: Backend Lead / DBA / Seguridad

Evidencia requerida: despliegue en staging, informe ZAP inicial, informe SonarCloud

6.3.2.Hito 2

MVP integrado y pruebas E2E superadas

Fecha: 28/03/2026

Responsable: PM / QA Lead

Evidencia requerida: E2E en Cypress con todos los flujos Must pasando; checklist de aceptación firmado por Product Owner

6.3.3.Hito 3

Sistema validado bajo carga y documentación entregada

Fecha:05/04/2026

Responsable: DevOps / PM

Evidencia requerida: informe k6 con umbrales cumplidos, documentación final subida y acta de aceptación

6.4 Plan de pruebas y aseguramiento de calidad

6.4.1.Estrategia de pruebas

- ☒ Unitarias
 - ✓ Cobertura de funciones críticas con Jest
 - ✓ Ejecución en CI por cada PR
- ☒ Integración
 - ✓ Supertest para endpoints
 - ✓ Pruebas contra base de datos de staging
- ☒ E2E
 - ✓ Cypress para flujos ciudadano, conductor y admin
 - ✓ Ejecutar en staging antes de release.
- ☒ Carga y rendimiento
 - ✓ K6 para escenarios de concurrencia
 - ✓ Definir scripts para 100, 500 y 1000 usuarios simulados.
- ☒ Seguridad
 - ✓ SonarCloud (SAST) y OWASP ZAP (DAST) integrados en pipeline
 - ✓ Corrección obligatoria de vulnerabilidades críticas.

6.4.2.Quality gates

Control	Umbral / Regla
Cobertura unitaria mínima	80% en módulos críticos
SonarCloud quality gate	No blocker; no critical issues on main
DAST (ZAP)	No critical/high vulnerabilities open on staging
Performance	API p95 $\leq$ 500 ms; route calc p95 $\leq$ 30 s for 500 items

6.5 Gestión de riesgos y plan de contingencia

6.5.1.Registro de riesgos

ID	Riesgo	Probabilidad	Impacto	Mitigación	Responsable
R1	Plazo insuficiente	Alta	Alto	Priorizar backlog Must; añadir recursos temporales; recortar alcance no crítico	PM
R2	Vulnerabilidades críticas en seguridad	Media	Alto	Integrar SAST/DAST desde PR; corrección inmediata; bloqueo de merges	Seguridad
R3	Problemas de rendimiento geoespacial	Media	Alto	Índices GIST; pruebas de carga tempranas; optimización	DBA/DevOps

				consultas	
R4	Camiones fuera de servicio / logística real	Baja	Medio	Reasignación automática; marcar intervención manual; notificaciones	Operaciones
R5	Dependencia de sensores reales (no integrados)	Alta	Medio	Simulación de datos; diseñar capa de abstracción para futura integración	PM/Dev

6.5.2. Planes de contingencia

Las acciones concretas por riesgo crítico son:

- ☑ R1 Contingencia (Plazo insuficiente)
 - ✓ Activar refuerzo temporal (1 senior backend + 1 junior frontend) por 2 semanas
 - ✓ Recortar entregables Should/May
 - ✓ Negociar extensión con Product Owner
- ☑ R2 Contingencia (Vulnerabilidades críticas)
 - ✓ Bloquear despliegue
 - ✓ Crear hotfix branch
 - ✓ Asignar equipo de seguridad y backend para corrección en 48 h
 - ✓ Ejecutar ZAP y SonarCloud tras fix
- ☑ R3 Contingencia (Rendimiento)
 - ✓ Degradación controlada de funcionalidades no críticas (p. ej. reducir frecuencia de actualización de mapas)
 - ✓ Escalar réplicas DB
 - ✓ Activar caché geoespacial

6.6 Gestión del Triángulo de Hierro

6.6.1. Opciones ante reducción de plazo

6.6.1.1. Opción A – Aumentar recursos

- ✓ Incorporar 2 desarrolladores adicionales durante 4 semanas.
- ✓ Impacto: +160 h
- ✓ Coste estimado aproximado €8.000 (estimación orientativa)

6.6.1.2. Opción B – Recortar alcance

- ✓ Eliminar entregables de prioridad Should y May (p. ej. panel BI, integraciones externas)
- ✓ Impacto: reduce 80-120 h
- ✓ Coste 0 adicional
- ✓ Menor funcionalidad

6.6.1.3. Comparativa

Aumentar recursos acelera entrega pero incrementa coste
 Recortar alcance mantiene coste pero reduce funcionalidad
 Decisión a tomar por Product Owner

6.7 Criterios de aceptación y checklist de entrega

6.7.1. Checklist - Lista de verificación para cada entregable

- ☑ Documentación
 RF/RNF, OpenAPI, DDL, diagramas UML, manuales subidos y enlazados
- ☑ Código

Repositorio con ramas limpias, PRs revisados, tests verdes

☒ Pruebas

Unitarias, integración y E2E con resultados; informe de rendimiento y seguridad

☒ Operaciones

Pipelines configurados, staging operativo, runbooks disponibles

☒ Firmas

acta de aceptación firmada por Product Owner y PM

6.8 Cierre del proyecto y transferencia

6.8.1. Informe final

El informe final incluirá desviaciones respecto a planificación y presupuesto, métricas de calidad y rendimiento, resumen de pruebas, vulnerabilidades detectadas y corregidas, y lecciones aprendidas. El informe se publicará en /docs/closure/ y se entregará al Product Owner para firma.

6.8.2. Transferencia

Entregables de transferencia: manuales (/docs/manuals/), runbooks (/ops/runbooks/), acceso al repositorio con permisos, credenciales de servicio en gestor seguro, plan de soporte y SLA.

☒ SLA propuesto

✓ Soporte inicial 3 meses

✓ Tiempo de respuesta crítico 4 h

✓ Resolución crítica 48 h

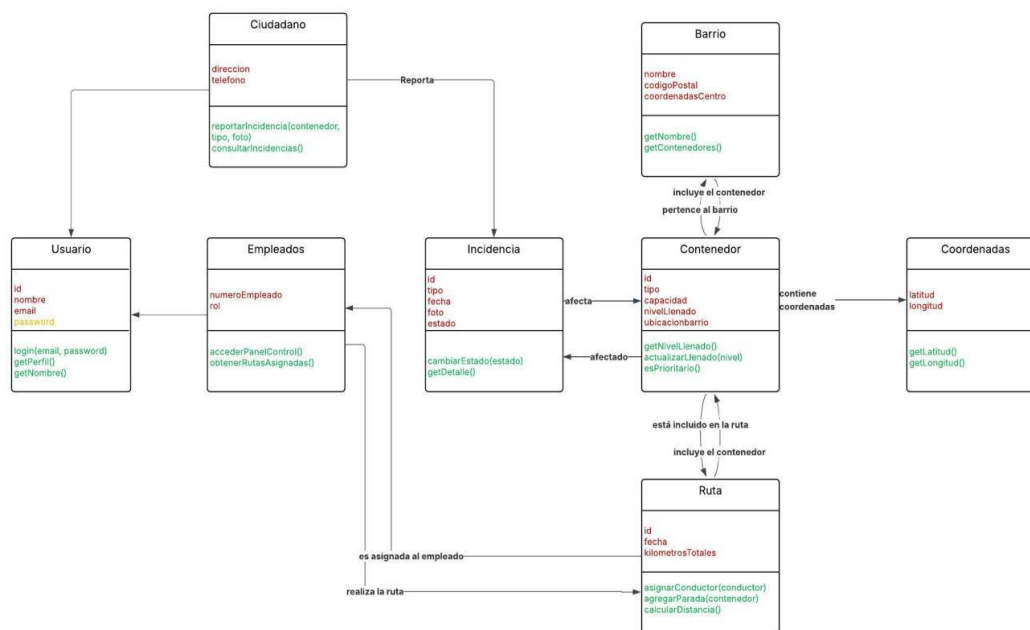
7 ANEXOS Y ARTEFACTOS

Esta sección reúne los anexos y artefactos que acompañan la memoria del proyecto. Incluye diagramas UML, pseudocódigo y pruebas del algoritmo de rutas, mockups, matrices auxiliares, cronogramas, ADRs, firmas y registros de aprobación.

7.1 Anexo A Diagramas UML

7.1.1. Clases

7.1.1.1. Diagrama de Clases

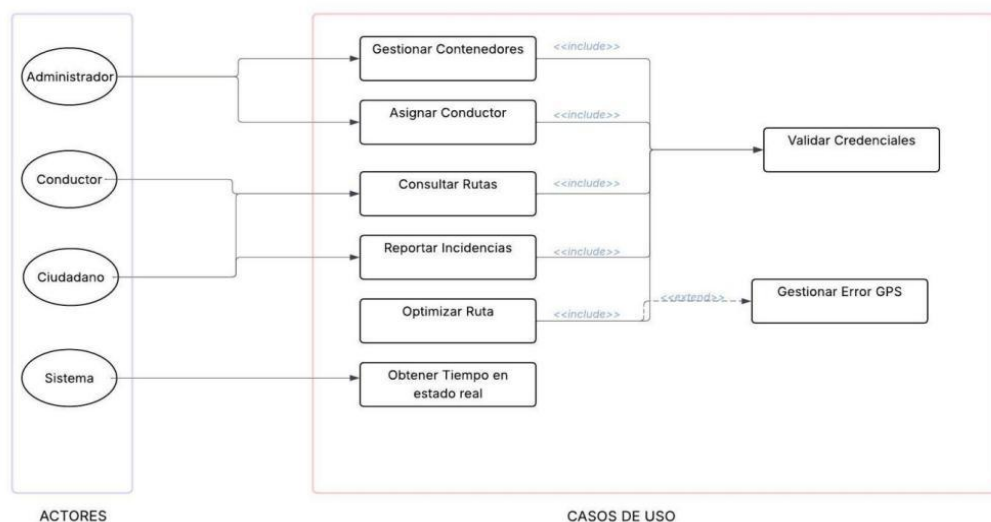


7.1.1.2. Leyenda del Diagrama de Clases

- ✓ Usuario (abstracto): id, nombre, email, teléfono, rol; métodos: autenticar(), actualizarPerfil().
- ✓ Ciudadano (Usuario): direccion; método: reportarIncidencia().
- ✓ Conductor (Usuario): licencia, camionesAsignados; métodos: recibirRuta(), marcarVaciado().
- ✓ Administrador (Usuario): permisos; métodos: asignarRuta(), gestionarUsuarios().
- ✓ Contenedor: id, codigo, tipo, geom, capacidad_litros, nivel_llenado, estado; métodos: getNivelLlenado(), actualizarNivel().
- ✓ Camion: id, matricula, capacidad_litros, estado; método: asignarRuta().
- ✓ Ruta: id, nombre, conductor, camion, fecha_programada, stops (ordenado); métodos: generarItinerario(), optimizarOrden().
- ✓ Incidencia: id, reportero, contenedor, tipo, descripcion, estado, foto_url, created_at, resuelto_at; método: cambiarEstado().
- ✓ Notificacion: id, usuario, tipo, payload, leida, created_at; método: marcarLeida().

7.1.2. Casos de uso

7.1.1.1 Diagrama de Casos de Uso

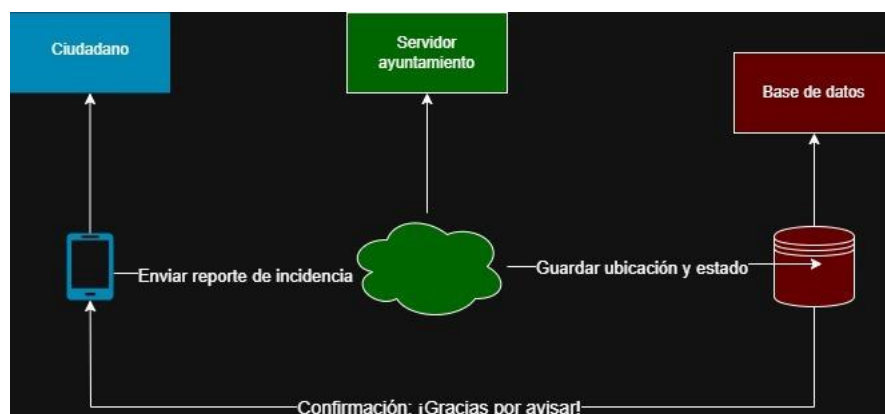


7.1.1.1 Tabla

Actor	Caso de uso	Descripción
Ciudadano	Reportar incidencia	Enviar foto, ubicación y descripción
Ciudadano	Consultar contenedores	Ver contenedores cercanos en mapa
Conductor	Recibir ruta	Recibir itinerario optimizado del día
Conductor	Marcar vaciado	Indicar que un contenedor ha sido vaciado
Administrador	Asignar ruta	Asignar o modificar rutas manualmente
Administrador	Gestionar usuarios	CRUD conductores y administradores

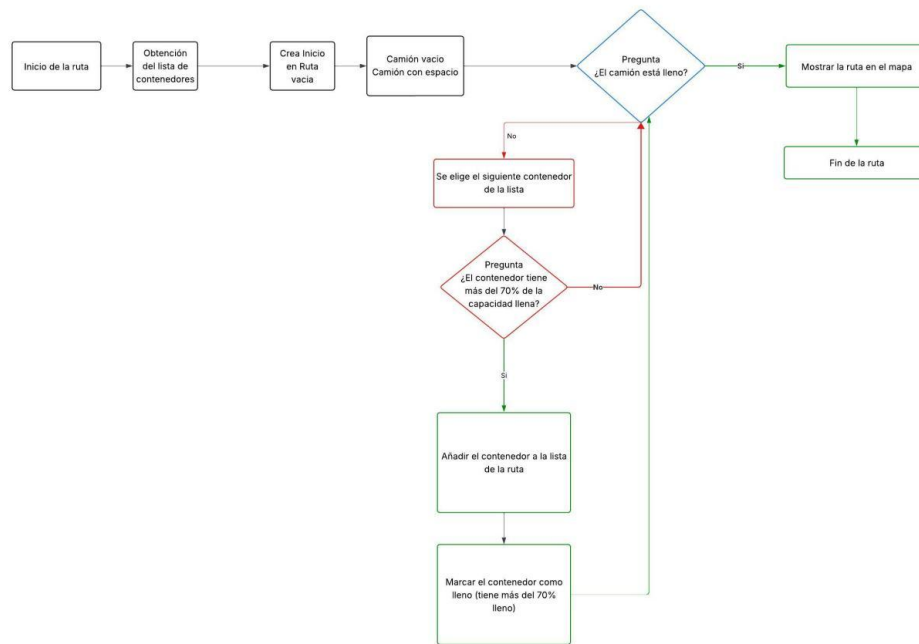
7.1.3. Secuencias

7.1.3.1. Diagramas de secuencia para reporte de incidencia



7.1.3.2. Diagramas de secuencia para generación de ruta

7.1.3.3. Diagramas de secuencia para marcación de vaciado



7.2 Anexo B Pseudocódigo y pruebas del algoritmo de rutas

7.2.1. Pseudocódigo completo

7.2.1.1. Algoritmo: Generación y optimización de rutas

Entrada:- containers: lista de contenedores con {id, geom, nivel_llenado, volumen_estimado, ventana_servicio, prioridad}- camiones: lista de camiones con {id, capacidad_litros, disponibilidad}- params: {UMBRAL_LLENADO, CAPACIDAD_CAMION, VENTANA_INICIO, VENTANA_FIN, TIEMPO_MAX_CALCULO}

Versión estructurada: FUNCION generar_rutas(containers, camiones, params):

```

  candidatos = FILTER(containers, c -> c.nivel_llenado >= params.UMBRAL_LLENADO AND
c.estado == 'activo')

```

```

  clusters = AGRUPAR_POR_ZONA(candidatos)

```

```

  rutas = []

```

PARA cada cluster EN clusters:

```

  lista = ORDENAR_POR_PRIORIDAD_Y_DISTANCIA(cluster)

```

MIENTRAS lista NO VACIA:

```

  ruta = NUEVA_RUTA()

```

```

  capacidad_restante = params.CAPACIDAD_CAMION

```

MIENTRAS lista NO VACIA Y capacidad_restante > 0:

```

  cont = POP_MEJOR_CANDIDATO(lista, ruta.ultima_posicion)

```

SI cont.volumen_estimado <= capacidad_restante Y RESPETA_VENTANA(cont, ruta):

```

  AÑADIR_PARADA(ruta, cont)

```

```

  capacidad_restante -= cont.volumen_estimado

```

```

  APLICAR_2OPT(ruta.paradas)

```

```

  ASIGNAR_CAMION_Y_CONDUCTOR(ruta, camiones)

```

```

  AÑADIR rutas, ruta

```

```

RETURN rutas

```

7.2.1.2. Versión heurística (resumen):

1. Seleccionar contenedores con nivel >= UMBRAL.
2. Para cada camión: construir ruta greedy eligiendo el siguiente contenedor más cercano y de mayor prioridad que quepa.
3. Aplicar 2-opt por ruta.
4. Si quedan contenedores, crear rutas adicionales o marcar intervención manual.

7.2.1.3. Comentarios:

- AGRUPAR_POR_ZONA puede implementarse con cuadrícula o DBSCAN.

- POP_MEJOR_CANDIDATO prioriza por (prioridad, nivel_llenado, proximidad). - 2-opt mejora orden pero es $O(m^2)$ por ruta; limitar iteraciones para control de tiempo.

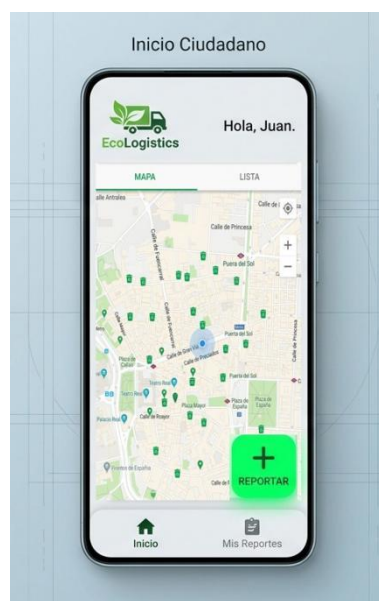
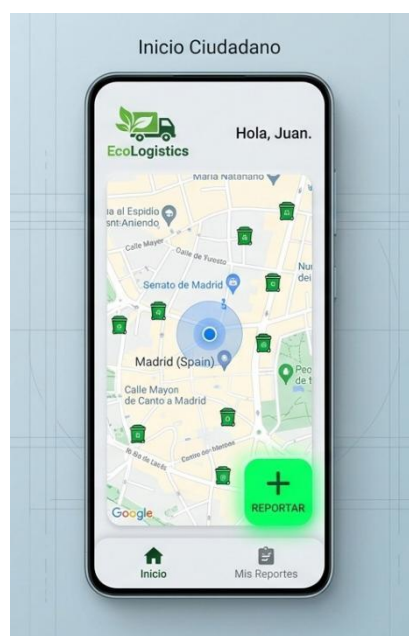
7.2.2. Casos de prueba

7.2.2.1. Tabla de casos de prueba

ID	Descripción	Datos de entrada (detallado)	Pasos	Resultado esperado	Criterio de aceptación
TC01	Ruta básica	10 contenedores; 1 camión 8000L; 4 contenedores nivel $\geq 70\%$	Ejecutar generación de rutas	Ruta asignada que incluye los 4 contenedores	Todos los $\geq 70\%$ incluidos
TC02	Capacidad insuficiente	50 contenedores; 1 camión 8000L	Ejecutar generación de rutas	Múltiples rutas creadas; ninguna excede capacidad	No exceder capacidad por ruta
TC03	Ventanas solapadas	Contenedores con ventanas 09:00-11:00 y 10:00-12:00	Ejecutar generación de rutas	Rutas respetan ventanas; contenedores fuera de ventana no asignados	Ninguna parada fuera de ventana
TC04	Alta prioridad	Mezcla prioridades; algunos alta	Ejecutar generación de rutas	Contenedores alta prioridad incluidos antes que media/baja	% alta prioridad cubierto $\geq 95\%$
TC05	Dispersión geográfica	Contenedores muy dispersos	Ejecutar generación de rutas y 2-opt	Distancia total razonable; 2-opt reduce recorrido	Reducción de distancia $\geq 10\%$ tras 2-opt
TC06	Falla de camión	3 camiones; 1 falla durante planificación	Ejecutar reasignación	Contenedores reasignados a otros camiones o marcados	Reasignación automática o flag manual

7.3 Anexo C Mockups

7.3.1. Capturas



Figuras 1 y 2 - Pantalla de inicio del ciudadano



Figura 3 - Formulario de incidencia (Ciudadano)

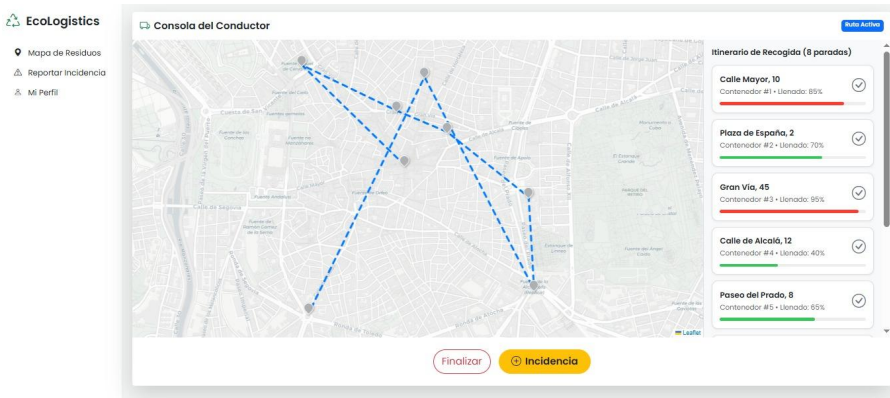


Figura 4 - Vista del Panel de Conductor (Sin llegada a Parada)

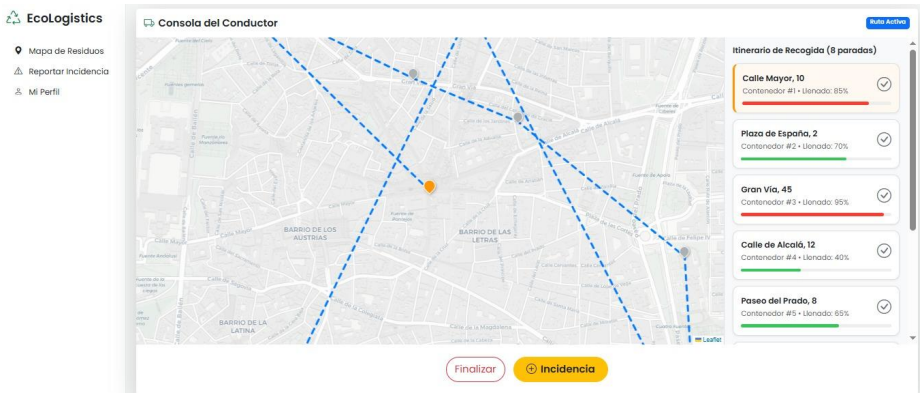


Figura 5 - Vista del Panel de Conductor (En Parada)

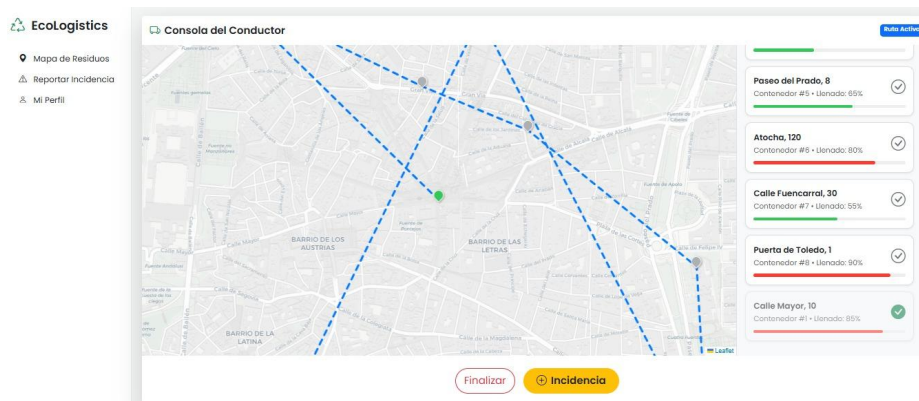


Figura 6 - Vista del Panel de Conductor (Parada Realizada)

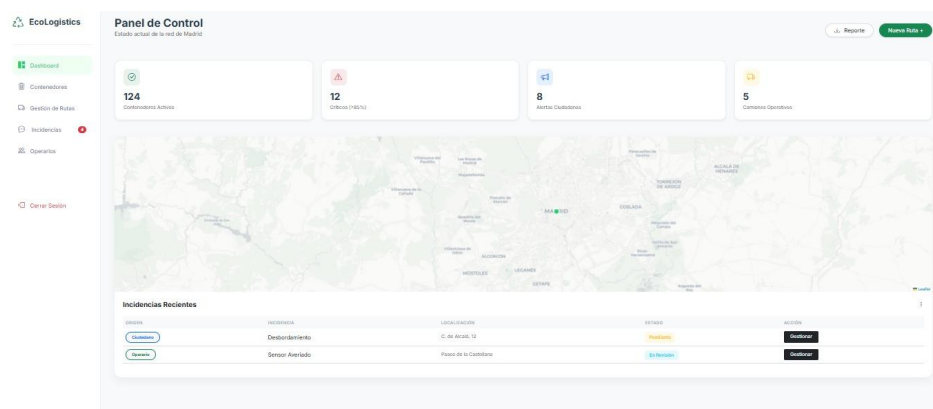


Figura 7 - Vista del Panel de Control del Administrador

7.4 Anexo D Matrices y tablas auxiliares

7.4.1. Matriz de trazabilidad

Descripción: matriz exportable que relaciona Requisitos ↔ Historias ↔ Casos de prueba ↔ Tests automatizados ↔ Entregables.

7.4.2. Matriz RACI

Actividad / Entregable	R (Responsable)	A (Aprobador)	C (Consultado)	I (Informado)
Planificación y seguimiento	PM	Product Owner	Lead Architect, BA	Equipo
Product Backlog y RF/RNF	BA	Product Owner	PM, Analysts	Equipo
Diseño de BBDD	DBA	Lead Architect	Backend Lead	PM
Contratos API	Backend Lead	Lead Architect	QA, Frontend	PM
Implementación Backend	Backend Team Lead	PM	DBA, QA	Equipo
Implementación Frontend	Frontend Lead	PM	UX, QA	Equipo
Pruebas y QA	QA Lead	PM	Devs	Product Owner
Despliegue y Operaciones	DevOps Lead	PM	DBA, Seguridad	Equipo
Entrega y aceptación	PM	Product Owner	Leads	Stakeholders

7.4.3.Stakeholders

Stakeholder	Tipo	Interés
Product Owner (Responsable Académico)	Interno	Representa la voz del cliente, prioriza requisitos, valida entregables, asegura MVP cumple objetivos funcionales y formativos establecidos
Jefe/a de Proyecto	Interno	Coordina el trabajo del equipo, se asegura de que el trabajo se entregue a tiempo, dentro del presupuesto, con la calidad acordada, y gestiona los riesgos.
Equipo de Frontend (UI/UX)	Interno	Diseñan una interfaz que resulte clara, accesible, coherente con los requisitos y agradable de usar.
Equipo de Backend (API)	Interno	Construyen una API REST sólida, segura, escalable y con buen rendimiento.
Equipo de Base de Datos y DevOps	Interno	Definen un modelo eficiente, optimizan las consultas y mantienen la infraestructura y los despliegues en buen estado
Equipo de QA / Testing	Interno	Realizan pruebas de todo tipo para garantizar la estabilidad y seguridad del sistema
Equipo de Documentación	Interno	Elaboran guías de usuario y manuales, y documentación técnica clara y estructurada
Ayuntamiento (cliente)	Externo	Busca optimizar la gestión de residuos, reducir los costes y mejorar la eficiencia del servicio
Ciudadanos	Externo	Consultan contenedores, reportan incidencias y contribuyen a la mejora del servicio
Conductores de camiones	Externo	Requieren una aplicación clara y usable durante la ruta, con recorridos eficientes
Administrador/gestor de flota	Externo	Gestiona rutas, conductores, contenedores e incidencias, y necesita datos fiables para la toma de decisiones
Personal de mantenimiento	Externo	Recibe avisos sobre contenedores dañados o incidencias que requieren intervención física.
Policía Local	Externo	Supervisa que los contenedores no generen riesgos, obstrucciones o problemas de seguridad vial
Servicios urbanos (limpieza viaria)	Externo	Buscan mejorar la percepción ciudadana y reducir el número de quejas
Alcaldía / Concejales	Externo	Interesados en mejorar la imagen del municipio, la satisfacción ciudadana y la eficiencia del servicio
Comunidad local / población afectada	Externo	percibe la calidad del servicio, la salubridad urbana y el impacto de la gestión de residuos.

7.5 Anexo E Cronogramas imprimibles

7.5.1. Gantt (del primer mes)



7.5.2. PERT

7.6 Firmas y aprobaciones

7.6.1. Plantilla de firmas

TÍTULO: Acta de Aceptación y Firmas

Proyecto: EcoLogistics Web – MVP

ID del entregable: _____

Versión: _____

Fecha de entrega: _____

Declaración de aceptación:

Por la presente, las partes abajo firmantes declaran haber revisado y aceptado el entregable identificado arriba, confirmando que cumple con los criterios de aceptación acordados y que se procede a su recepción formal.

Rol	Nombre	Organización	Firma	Fecha
Project Manager (PM)	_____	_____	_____	___ / ___ /
	_____	_____	_____	_____
	_____	_____	_____	___ / ___ /
Lead Técnico	_____	_____	_____	___ / ___ /
	_____	_____	_____	_____
	_____	_____	_____	___ / ___ /
Product Owner / Cliente	_____	_____	_____	___ / ___ /
	_____	_____	_____	_____
	_____	_____	_____	___ / ___ /
Responsable QA	_____	_____	_____	___ / ___ /
	_____	_____	_____	_____
	_____	_____	_____	___ / ___ /
Responsable Seguridad	_____	_____	_____	___ / ___ /
	_____	_____	_____	_____
	_____	_____	_____	___ / ___ /

Observaciones (campo libre)

Confirmación final

Con la firma del Product Owner / Cliente se da por aceptado el entregable y se autoriza su publicación/puesta en producción según corresponda.

7.6.2. Registro de aprobaciones

Fecha	Entregable	Aprobador	Observaciones
	Project Charter MVP (staging) Informe final		